

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO GIỮA KỲ PROJECT I

ĐỀ TÀI:

**TÌM HIỂU VỀ CÂY ĐỎ-ĐEN (RED-BLACK TREE)
VÀ ỨNG DỤNG THỰC TIỄN**

Họ tên: Lộc Thị Vân Anh

MSSV: 20215523

Kỳ: 20251

Hướng dẫn: PGS.TS. Nguyễn Khanh Văn

MỤC LỤC

LỜI MỞ ĐẦU.....	2
I. ĐẶT VẤN ĐỀ.....	3
1. Binary Search Tree.....	3
2. Vấn Đề Mất Cân Bằng	4
II. ĐỊNH NGHĨA VỀ CÂY ĐỎ ĐEN.....	5
III. CẤU TRÚC VÀ TÍNH CHẤT.....	5
1. Chiều Cao Đen Của Một Node	5
1.1. Khái Niệm	5
1.2. Ví Dụ Minh Hoạ	6
1.3. Vai Trò Của Chiều Cao Đen	6
1.4. Hệ Quả.....	7
2. Cấu Trúc Của Một Node.....	7
IV. THAO TÁC CƠ BẢN.....	8
1. SEARCH (Tìm Kiếm)	8
2. INSERTION (Thêm Node)	10
2.1. TH1: N được chèn vào vị trí node gốc của cây	11
2.2. TH2: P là node đen.....	12
2.3. TH3: Cả P và U đều là node đỏ.....	13
3. DELETE (Xoá)	24
3.1. TH1: X màu đỏ hoặc nút gốc.....	25
3.2. TH2: X là node đen và có 1 con là nút đỏ.....	26
3.3. TH3: X là node đen và có 2 con là nút đen.....	28
*Ví dụ về Delete:	37
V. ỨNG DỤNG THỰC TIỄN.....	38
1. Cấu Trúc Dữ Liệu Trong Thư Viện Lập Trình Chuẩn (Standard Libraries).....	38
2. Hệ Điều Hành Và Hệ Thống Nhân Kernel.....	41
3. Cơ Sở Dữ Liệu Và Hệ Thống Lưu Trữ.....	41
4. Mạng Máy Tính Và Bảng Định Tuyến	41
5. Trình Biên Dịch Và Phân Tích Mã Nguồn.....	42
6. Trình Duyệt Web Và Công Cụ Render	42
VI. KẾT LUẬN.....	42
VII. KẾ HOẠCH GIAI ĐOẠN 2.....	43
VIII. TÀI LIỆU THAM KHẢO	45

LỜI MỞ ĐẦU

Trong lĩnh vực **khoa học máy tính**, việc tổ chức và truy xuất dữ liệu một cách nhanh chóng, ổn định và hiệu quả luôn là một trong những vấn đề trọng tâm. Cùng với sự phát triển của công nghệ và quy mô dữ liệu, yêu cầu về các **cấu trúc dữ liệu có khả năng thích ứng, duy trì hiệu năng ổn định trong quá trình cập nhật** ngày càng trở nên quan trọng. Một trong những cấu trúc cơ bản và phổ biến nhất là **Binary Search Tree (BST)** – cây tìm kiếm nhị phân. Tuy nhiên, nhược điểm của BST là **đễ bị mất cân bằng** nếu dữ liệu được chèn theo thứ tự nhất định, dẫn đến hiệu năng suy giảm nghiêm trọng – độ phức tạp các thao tác có thể tăng từ $O(\log n)$ lên $O(n)$. Điều này tạo nên nhu cầu về các **cây nhị phân tự cân bằng (Self-Balancing Binary Search Trees)** nhằm đảm bảo hiệu quả tính toán ổn định hơn.

Trong số các phương pháp tự cân bằng được đề xuất, **Red-Black Tree** là một trong những thuật toán nổi bật nhờ **sự cân bằng giữa tính hiệu quả và độ phức tạp cài đặt**. Cấu trúc này không yêu cầu cân bằng tuyệt đối như cây AVL, nhưng vẫn đảm bảo chiều cao của cây luôn ở mức logarit so với số lượng phần tử. Cơ chế sử dụng **hai màu (đỏ – đen)** cùng với các phép **xoay (rotation)** giúp cây tự điều chỉnh trạng thái mỗi khi có sự thay đổi dữ liệu, duy trì được độ cân bằng tương đối mà không tốn quá nhiều chi phí xử lý. Việc lựa chọn **Red-Black Tree** làm chủ đề nghiên cứu xuất phát từ **tính phổ quát, giá trị lý thuyết và ứng dụng nền tảng của nó** trong các lĩnh vực thuật toán và cấu trúc dữ liệu. Red-Black Tree là ví dụ tiêu biểu thể hiện rõ mối quan hệ giữa **tính cân bằng, hiệu năng và độ phức tạp của thuật toán**, đồng thời là cơ sở để hiểu sâu hơn về các hệ thống quản lý dữ liệu hiện đại.

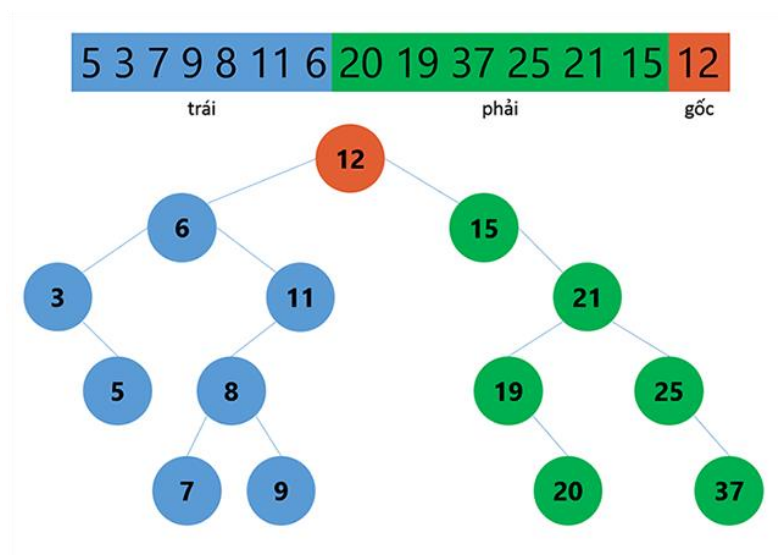
Mục tiêu của báo cáo là **phân tích nguyên lý hoạt động, cấu trúc và các thuật toán chính** của Red-Black Tree (tìm kiếm, chèn, xóa), đồng thời làm rõ **vai trò của cơ chế cân bằng động** trong việc tối ưu hóa các thao tác trên cây nhị phân. Qua đó, người đọc có thể nhận thức được **ý nghĩa của Red-Black Tree trong việc duy trì hiệu năng ổn định và bền vững của hệ thống dữ liệu**, cả trong lý thuyết lẫn trong các ứng dụng thực tế quy mô lớn.

I. ĐẶT VẤN ĐỀ

Trong lĩnh vực **khoa học máy tính**, tốc độ truy xuất và quản lý dữ liệu ảnh hưởng trực tiếp đến hiệu năng của hệ thống. Các cấu trúc dữ liệu tuyến tính như **array** hay **linked list** có độ phức tạp tìm kiếm trung bình là $O(n)$, khiến chúng không phù hợp cho những ứng dụng yêu cầu truy xuất nhanh trên tập dữ liệu lớn.

Cấu trúc **Binary Search Tree (BST)** ra đời như một giải pháp tối ưu hơn, cho phép tìm kiếm, thêm hoặc xóa phần tử trong $O(\log n)$ khi cây được cân bằng. Đây chính là **bước tiến đầu tiên** để tối ưu hóa việc lưu trữ và tìm kiếm dữ liệu.

1. Binary Search Tree



Hình 1.1. Binary Search Tree

Binary Search Tree (BST) là một cấu trúc dữ liệu dạng cây, trong đó mỗi node có tối đa hai node con: **left child** và **right child**.

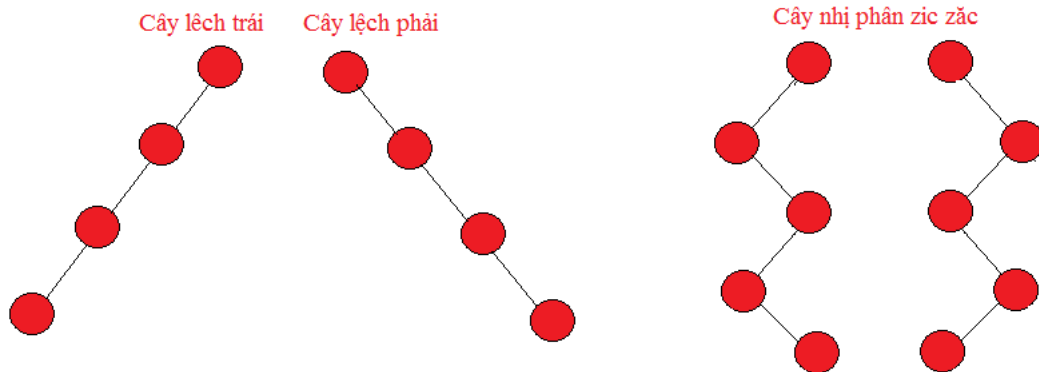
Mỗi node thỏa mãn điều kiện:

- Mọi node trong cây con bên trái có giá trị **nhỏ hơn** node cha.
- Mọi node trong cây con bên phải có giá trị **lớn hơn** node cha.

Cấu trúc này cho phép tìm kiếm, chèn, và xóa phần tử một cách hiệu quả nhờ tính chất chia để trị.

2. Vấn Đề Mất Cân Bằng

Mặc dù BST có thể đạt hiệu suất tối ưu $O(\log n)$, nhưng nếu dữ liệu được chèn theo thứ tự tăng hoặc giảm dần, BST sẽ bị **mất cân bằng** và trở nên kém hiệu quả, có thể đạt độ phức tạp xấu nhất là $O(n)$.



Hình 1.2. Ví dụ về cây suy biến

Nếu có một cây nhị phân cân bằng hoàn toàn, **chi phí cho việc tìm kiếm** xấp xỉ $O(\log_2 n)$

VD: 1 triệu node $\Rightarrow$ chi phí tìm kiếm $= \log_2 1.000.000 \sim 20$

Phương pháp thêm trên cây TKNP, có thể có những biến dạng mất cân đối nghiêm trọng, **chi phí cho việc tìm kiếm** chỉ xấp xỉ $O(n)$

VD: 1 triệu node $\Rightarrow$ chi phí tìm kiếm $= n = 1.000.000$

Khi nhận thấy những hạn chế của cây tìm kiếm nhị phân thông thường, các nhà nghiên cứu đã đề xuất nhiều **cấu trúc cây tự cân bằng (self-balancing binary search tree)** với mục tiêu duy trì chiều cao của cây ở mức xấp xỉ logarit.

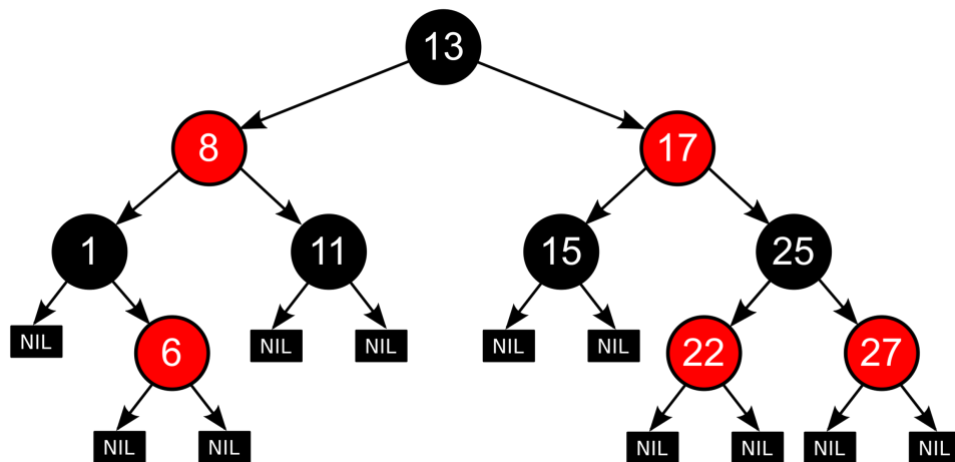
Các phương pháp tiêu biểu như **AVL Tree**, **Splay Tree** hay **Red-Black Tree** đều áp dụng những cơ chế riêng để tái cân bằng cây sau mỗi thao tác chèn hoặc xóa.

Trong đó, **Red-Black Tree** được xem là một giải pháp tối ưu nhờ sự **kết hợp hài hòa giữa hiệu năng và tính đơn giản trong cài đặt**. Cấu trúc này không yêu cầu cân bằng tuyệt đối như AVL Tree, nhưng vẫn đảm bảo độ cao của cây nằm trong giới hạn cho phép, giúp mọi thao tác cơ bản được thực hiện với thời gian $O(\log n)$ một cách ổn định.

II. ĐỊNH NGHĨA VỀ CÂY ĐỎ ĐEN

Red-Black Tree là một **Binary Search Tree** có thêm **thuộc tính màu (color)** để đảm bảo sự cân bằng tương đối của cây. Mỗi node trong cây có màu **Red** hoặc **Black**, và toàn bộ cây tuân theo 5 quy tắc chính (red-black properties):

1. Mỗi node được tô màu đỏ hoặc đen.
2. Node gốc (**root**) luôn có màu đen.
3. Các node lá (NIL hoặc NULL) được xem là đen.
4. Nếu một node là đỏ thì hai node con của nó đều là đen.
5. Mọi đường đi từ một node đến các node lá của nó đều chứa cùng số lượng node đen (gọi là **black height**).



Hình II.1. Red-black Tree

III. CẤU TRÚC VÀ TÍNH CHẤT

1. Chiều Cao Đen Của Một Node

1.1. Khái Niệm

Trong cây đỏ-đen, mỗi node được gán một thuộc tính màu — **đỏ (Red)** hoặc **đen (Black)**. Khái niệm **chiều cao đen (Black Height)** được dùng để đo lường **độ cân bằng của cây**, là yếu tố cốt lõi giúp Red-Black Tree duy trì hiệu năng ổn định trong mọi thao tác.

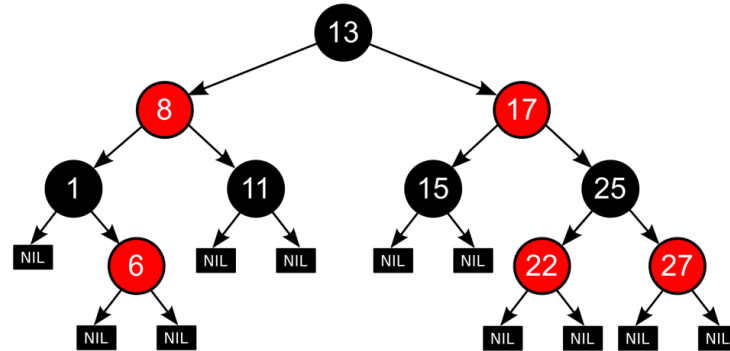
⇒ **ĐỊNH NGHĨA:** Chiều cao đen của một node X là số lượng node màu đen trên đường đi từ X đến bất kỳ node lá NIL (NULL), không tính node lá NIL ở cuối đường.

Nói cách khác, nếu ta bắt đầu từ node X và đi xuống dưới theo mọi nhánh cho đến các lá NIL, thì tất cả các đường đi này **đều chứa cùng số lượng node màu đen**. Tính chất

này là một trong **năm quy tắc cơ bản** của Red-Black Tree và là yếu tố quyết định để đảm bảo rằng **chiều cao của cây không vượt quá $2 \times \log_2(n + 1)$** .

1.2. Ví Dụ Minh Hoạ

Ta xét ví dụ ở hình II.1. như ở trên:



VD: Xét node 13:

13 - 8 - 1 (2 node đen)

13 - 8 - 1 - 6 (2 node đen)

13 - 8 - 11 (2 node đen)

13 - 17 - 15 (2 node đen)

13 - 17 - 25 (2 node đen)

13 - 17 - 25 - 22 (2 node đen)

13 - 17 - 25 - 27 (2 node đen)

⇒ Chiều cao đen của node 13 là 2.

1.3. Vai Trò Của Chiều Cao Đen

Chiều cao đen giữ vai trò đặc biệt quan trọng trong việc duy trì tính cân bằng của Red-Black Tree:

1. Bảo đảm độ cân bằng tương đối:

Mọi đường đi từ một node đến lá đều có cùng số node đen → cây không thể nghiêng quá nhiều về một phía.

2. Giới hạn chiều cao tối đa của cây:

Dựa vào chiều cao đen, có thể chứng minh rằng cây đỏ-đen với n node có chiều cao không vượt quá $2 \times \log_2(n + 1)$.

3. Là cơ sở cho các phép chứng minh độ phức tạp:

Các phân tích về thời gian thực thi của Red-Black Tree (tìm kiếm, thêm, xóa) đều dựa trên tính chất chiều cao đen để suy ra rằng tất cả các thao tác đều có độ phức tạp $O(\log n)$.

Khái niệm “**chiều cao đen**” là công cụ giúp **định lượng hóa trạng thái cân bằng** của Red-Black Tree mà không cần cây phải tuyệt đối cân bằng như AVL. Nhờ có quy tắc

này, Red-Black Tree duy trì được tính ổn định trong khi vẫn hạn chế được số lượng phép xoay cần thiết, giúp nó trở thành **một trong những cấu trúc dữ liệu tự cân bằng hiệu quả và phổ biến nhất hiện nay**.

1.4. Hệ Quả

Từ tính chất đường đi của một nút tới tất cả các nút lá đều có chiều cao đen bằng nhau, suy ra được hệ quả: Đường đi dài nhất từ gốc tới lá không dài quá **2 lần** đường đi ngắn nhất từ gốc tới lá.

Ta có thể hiểu điều này trực quan như sau:

Giả sử một đường đi ngắn nhất từ gốc đến lá chỉ bao gồm các node màu **đen** $\rightarrow$ tổng số node trên đường đó là h .

- Đường đi dài nhất có thể xen kẽ giữa node đỏ và node đen, nhưng do **quy tắc cấm hai node đỏ liên tiếp**, số node tối đa trên đường dài nhất sẽ là **2h** (các node đỏ “chèn xen kẽ” giữa các node đen).

$\Rightarrow$ Như vậy, cây đỏ-đen luôn duy trì được **chiều cao giới hạn trong khoảng logarit**, đảm bảo mọi thao tác (tìm kiếm, thêm, xóa) đều có độ phức tạp **$O(\log n)$** .

2. Cấu Trúc Của Một Node

Trong cây đỏ-đen, mỗi node (nút) được thiết kế để lưu trữ thông tin và thể hiện mối quan hệ phân cấp giữa các phần tử trong cây. Khác với cây nhị phân tìm kiếm thông thường, node trong Red-Black Tree ngoài các thành phần cơ bản như dữ liệu và các con trỏ liên kết còn có thêm một thuộc tính đặc trưng là **màu sắc**, giúp cây duy trì được trạng thái cân bằng sau mỗi lần thêm hoặc xóa phần tử.

Cụ thể, một node trong cây đỏ-đen bao gồm các thành phần chính:

- **Giá trị dữ liệu (Info):** Trường **Info** chứa giá trị hoặc khóa (key) của phần tử, dùng làm cơ sở cho việc so sánh trong các thao tác tìm kiếm, chèn hoặc xóa.
- **Con trỏ tới node con bên trái (Left)**
- **Con trỏ tới node con bên phải (Right)**
 - $\Rightarrow$ Hai trường **Left** và **Right** là các con trỏ giúp kết nối node hiện tại với các node con của nó — node con trái luôn mang giá trị nhỏ hơn, còn node con phải mang giá trị lớn hơn node hiện tại, đảm bảo duy trì tính chất của cây tìm kiếm nhị phân.
- **Con trỏ tới node cha (Parent):** Trường **Parent** cho phép node biết được vị trí của mình trong cấu trúc cây, từ đó hỗ trợ quá trình duyệt ngược lên khi cần thực hiện các phép xoay (rotation) hoặc điều chỉnh sau khi thêm và xóa node.
- **Thuộc tính màu (Color):** Thuộc tính **Color** mang giá trị là **RED** (đỏ) hoặc **BLACK** (đen), thể hiện màu của node và đóng vai trò quan trọng trong việc kiểm soát trạng thái cân bằng của toàn bộ cây.

Trong lập trình, cấu trúc của một node cây đỏ-đen có thể được mô tả như sau:

```
enum COLOR {
    RED,
    BLACK
};

struct RBNode {
    int Info;           // Giá trị (key) của node
    RBNode* Left;       // Con trỏ đến node con bên trái
    RBNode* Right;      // Con trỏ đến node con bên phải
    RBNode* Parent;     // Con trỏ đến node cha
    COLOR Color;        // Màu của node (RED hoặc BLACK)
};
```

Trong đó:

- kiểu liệt kê `enum COLOR` quy định hai giá trị màu hợp lệ của node là **RED** và **BLACK**.
Cấu trúc `RBNode` chứa đầy đủ các thông tin cần thiết để biểu diễn một node trong cây đỏ-đen.
- Việc bổ sung thêm thuộc tính `Color` tuy chỉ làm tăng kích thước lưu trữ không đáng kể, nhưng lại giúp cây có khả năng **tự cân bằng động** — điều mà cây nhị phân thông thường không có được.
- Các con trỏ `Left`, `Right` và `Parent` cho phép dễ dàng truy cập tới các node liên quan, đồng thời giúp thực hiện các phép biến đổi cấu trúc như **Left Rotation** hoặc **Right Rotation** trong quá trình duy trì các tính chất của cây.

Nhìn chung, cấu trúc của một node trong cây đỏ-đen được thiết kế **đơn giản, rõ ràng và hiệu quả**. Chỉ với việc thêm một biến màu, cây đỏ-đen có thể theo dõi trạng thái cân bằng và thực hiện các điều chỉnh cần thiết sau mỗi thao tác, đảm bảo cho tất cả các phép chèn, xóa và tìm kiếm đều hoạt động trong thời gian **$O(\log n)$** .

Chính thiết kế tối giản nhưng tinh tế này đã giúp Red-Black Tree trở thành một trong những cấu trúc dữ liệu nền tảng và bền vững nhất trong lĩnh vực khoa học máy tính hiện đại.

IV. THAO TÁC CƠ BẢN

Cây đỏ đen cũng có các thao tác giống như cây nhị phân tìm kiếm

1. SEARCH (Tìm Kiếm)

Red-Black Tree tuân thủ **tính chất BST**: với mọi nút x , mọi khóa trong cây con trái $< x \rightarrow \text{Info}$, mọi khóa trong cây con phải $> x \rightarrow \text{Info}$.

Vì vậy, thao tác **Search** của RBT giống hệt BST, **không đụng đến màu** và **không cần xoay**. Thời gian chạy phụ thuộc vào chiều cao cây và do RBT giữ chiều cao xấp xỉ logarit nên:

Độ phức tạp: $T(n) = O(h) = O(\log n)$

Thuật toán đệ quy:

```
RBNode* search(RBNode* root, int key) {
    if (root == NULL) return NULL;           // Cây rỗng / tới lá NIL
    if (key == root->Info) return root;      // Tìm thấy
    if (key < root->Info)                     // Nhỏ hơn -> đi trái
        return search(root->Left, key);
    else                                     // Lớn hơn -> đi phải
        return search(root->Right, key);
}
```

Đặc điểm:

- Ngắn gọn, dễ hiểu.
- Phù hợp khi chiều cao cây nhỏ (logarit) $\rightarrow$ độ sâu đệ quy an toàn.

Thuật toán không đệ quy (lặp):

```
RBNode* search(RBNode* root, int key) {
    RBNode* p = root;
    while (p != NULL) {
        if (key == p->Info) return p;
        if (key < p->Info) p = p->Left;
        else               p = p->Right;
    }
    return NULL;           // Không tìm thấy
}
```

Đặc điểm:

- Tránh overhead lời gọi hàm đệ quy.
- Thường dùng trong cài đặt thư viện/hệ thống.

Trường hợp biên & lưu ý triển khai:

- **Cây rỗng / đến lá NIL**: trả NULL.
- **Trùng khóa**: với cây lưu **bộ khóa duy nhất**, trả ngay node hiện tại; nếu cho phép **bản sao (duplicates)**, cần quy ước (ví dụ: chèn bản sao vào **nhánh phải** và khi tìm thì có thể tìm **first/last** bằng cách đi tiếp sang trái/phải khi bằng).
- **So sánh khóa phức tạp**: nếu Info là cấu trúc (string, pair, record), cần truyền **Comparator**.

- **Bảo toàn RBT:** Search **không thay đổi màu, không xoay** → không làm ảnh hưởng các thuộc tính đỏ-đen.

2. INSERTION (Thêm Node)

Trong cây đỏ-đen, thao tác **chèn (Insertion)** không chỉ đơn thuần là thêm một node mới vào vị trí thích hợp như trong cây nhị phân tìm kiếm, mà còn phải **duy trì các tính chất đỏ-đen** sau khi phần tử được thêm vào.

Điều này là cần thiết vì việc thêm một node mới có thể **phá vỡ sự cân bằng màu sắc hoặc cấu trúc** của cây, làm vi phạm một hoặc nhiều quy tắc cơ bản (ví dụ: “node đỏ không có con đỏ”, “root luôn là đen”, “mọi đường đi từ một node đến các lá đều có cùng số node đen”).

Quá trình chèn một node vào Red-Black Tree thường được chia thành **hai giai đoạn chính**:

1. Chèn như trong cây nhị phân tìm kiếm (BST):

Node mới được thêm vào vị trí phù hợp dựa trên giá trị khóa, tuân theo quy tắc:

- Nếu giá trị nhỏ hơn node hiện tại → đi sang trái.
- Nếu giá trị lớn hơn → đi sang phải.

2. Cân bằng lại cây (Fix-up):

Sau khi chèn, cây có thể vi phạm tính chất đỏ-đen. Khi đó cần thực hiện **đổi màu và/hoặc xoay cây (rotation)** để khôi phục cân bằng.

Thông thường, node mới được **tô màu đỏ** ngay khi chèn để không làm tăng số lượng node đen trên bất kỳ đường đi nào.

Sau đó, tùy vào màu của node cha và node chú, cây sẽ được điều chỉnh theo **ba nhóm trường hợp chính (case 1, 2, 3)** để đảm bảo mọi quy tắc đỏ-đen được khôi phục.

Nhờ cơ chế kết hợp giữa tô màu và phép xoay, cây đỏ-đen có thể **tự động duy trì cân bằng động** sau mỗi lần thêm phần tử mà không cần tính toán độ cao như ở cây AVL.

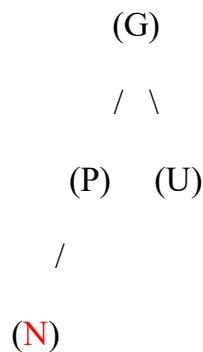
Kết quả là, thao tác chèn vẫn luôn có độ phức tạp **$O(\log n)$** , đảm bảo hiệu năng ổn định ngay cả với dữ liệu lớn hoặc thứ tự chèn bất kỳ.

Quy ước gọi tên trong quá trình chèn

Trước khi đi vào chi tiết các bước xử lý trong quá trình chèn, ta cần xác định một số **quy ước đặt tên cho các node** nhằm thuận tiện cho việc mô tả và minh họa thuật toán. Các quy ước này được sử dụng nhất quán trong tất cả các tài liệu và mô phỏng về Red-Black Tree.

- **N (New Node):** Là node vừa được thêm vào cây. Mặc định, node này được tô màu đỏ ngay khi được chèn, để không làm thay đổi số lượng node đen trên các đường đi.
- **P (Parent):** Là node cha của N, tức là node mà N được gắn vào làm con trái hoặc con phải tùy theo giá trị khóa.
- **U (Uncle):** Là node chú hoặc bác của N, nghĩa là node cùng cấp với P và có chung cha với P (tức là node con còn lại của ông G).
- **G (Grandparent):** Là node ông của N, tức là cha của node P.

Các ký hiệu trên được minh họa như sau:



Trong đó:

- N là node mới được chèn.
- P là cha của N.
- G là ông của N.
- U là anh em của P, tức là chú/bác của N.

Những ký hiệu này được sử dụng xuyên suốt trong phần trình bày tiếp theo để mô tả ba trường hợp xử lý cân bằng (**Insertion Fix-up**), giúp việc diễn giải thuật toán trở nên rõ ràng và trực quan hơn. Việc đặt tên thống nhất cũng hỗ trợ việc trình bày các phép xoay (**rotation**) và thay đổi màu (**recoloring**) trong các tình huống vi phạm quy tắc đỏ-đen sau khi chèn node mới.

Sau khi node mới được thêm vào, có thể xảy ra nhiều tình huống khác nhau tùy thuộc vào màu của node cha (P) và node chú (U). Thuật toán chèn trong Red-Black Tree xử lý những tình huống đó thông qua bốn nhóm trường hợp chính.

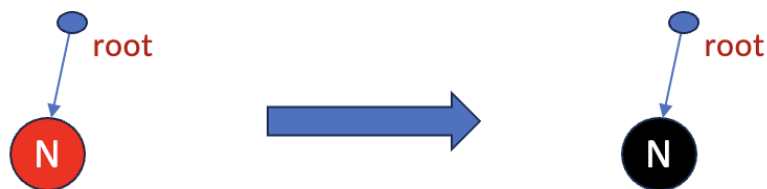
2.1. TH1: N được chèn vào vị trí node gốc của cây

Khi thực hiện thao tác chèn vào cây đỏ-đen, node mới (ký hiệu **N**) ban đầu luôn được tô **màu đỏ** nhằm không làm thay đổi số lượng node đen trên các đường đi. Tuy nhiên, trong trường hợp đặc biệt, **nếu node N được chèn vào khi cây đang rỗng**, tức là **N trở thành node gốc (root)**, thì màu đỏ của node này sẽ **vi phạm quy tắc đầu tiên** của cây đỏ-đen:

⇒ **Quy tắc 2:** Node gốc (root) của cây luôn phải có **màu đen (Black)**.

Do đó, khi phát hiện node **N** là root, thuật toán cần **sửa chữa vi phạm (fix-up)** bằng cách **đổi màu node N từ đỏ sang đen**. Việc đổi màu này đảm bảo rằng cây vẫn thỏa mãn toàn bộ các tính chất đỏ-đen ngay sau khi chèn phần tử đầu tiên.

Quá trình này không yêu cầu bất kỳ phép xoay (rotation) hay tái cấu trúc nào, vì tại thời điểm đó cây chỉ có duy nhất một node. Hành động đổi màu đơn giản này được coi là **bước khởi tạo hợp lệ cho một cây đỏ-đen mới**.



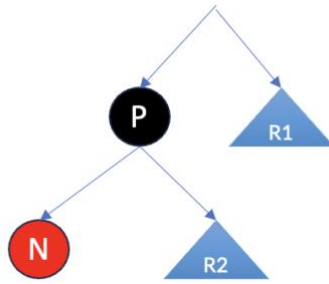
Hình IV.2.1. TH1: N được chèn vào gốc

Biểu diễn bằng hình minh họa:

- Node **N** (vừa được thêm vào) ban đầu có màu **đỏ**.
- Sau khi kiểm tra, thuật toán nhận thấy **N là root**, do đó tiến hành **đổi màu N thành đen** để khôi phục tính hợp lệ của cây.

Kết luận: Trường hợp 1 là tình huống đơn giản nhất trong quá trình chèn. Cây sau khi xử lý không cần xoay hay đổi cấu trúc, chỉ cần điều chỉnh lại màu của node gốc. Sau bước này, cây trở thành một **Red-Black Tree hợp lệ**, đáp ứng đầy đủ năm quy tắc đỏ-đen.

2.2. TH2: P là node đen



Hình IV.2.2. TH2: P là node đen

Trong quá trình chèn một node mới (N) vào cây đỏ-đen, sau khi N được thêm vào và tô màu đỏ, ta cần kiểm tra xem việc chèn này có làm **vi phạm các tính chất đỏ-đen** hay không.

Trường hợp thứ hai xảy ra khi **node cha (P) của N có màu đen**.

Khi đó, ta nhận thấy rằng cấu trúc của cây **vẫn hoàn toàn hợp lệ**, vì:

1. Không vi phạm định nghĩa của cây đỏ-đen:

Theo quy tắc, “mọi node đỏ đều phải có con là node đen”.

Trong tình huống này, node N là đỏ và node cha P là đen, nên điều kiện này vẫn được thỏa mãn.

2. Không ảnh hưởng đến chiều cao đen của cây:

Số lượng node đen trên mọi đường đi từ gốc đến lá **không thay đổi**, vì node N mới thêm vào là node đỏ, không làm tăng số lượng node đen nào trên bất kỳ đường đi nào.

Do đó, việc chèn node mới **không gây ra bất kỳ vi phạm nào** trong năm tính chất cơ bản của Red-Black Tree.

Thuật toán **không cần thực hiện bước cân bằng (Fix-up)** nào khác, và cây vẫn được xem là hợp lệ ngay sau khi chèn.

2.3. TH3: Cả P và U đều là node đỏ

Khi thực hiện thao tác chèn một node mới (N) vào cây đỏ-đen, node N luôn được tô màu đỏ ngay sau khi thêm vào. Trong trường hợp này, nếu **node cha (P)** và **node chú (U)** của N **đều có màu đỏ**, thì cấu trúc cây sẽ **vi phạm quy tắc số 4 của cây đỏ-đen**:

⇒ **Quy tắc 4:** Mọi node đỏ đều phải có cha là node đen.

Do cả P và U đều là đỏ, nên node ông (G) của N (cha của P và U) chắc chắn là **node đen**.

Khi đó, tồn tại liên tiếp hai node đỏ (P và N), làm sai lệch quy tắc trên và đòi hỏi thuật toán phải thực hiện bước **hiệu chỉnh màu (recoloring)** để khôi phục lại tính cân bằng màu sắc của cây.

Bước xử lý trong trường hợp này:

1. Đổi màu P và U từ đỏ sang đen:

Việc này giúp loại bỏ vi phạm “liên tiếp hai node đỏ”, vì sau khi đổi, P và U trở thành node đen – không còn nằm cạnh node đỏ nào.

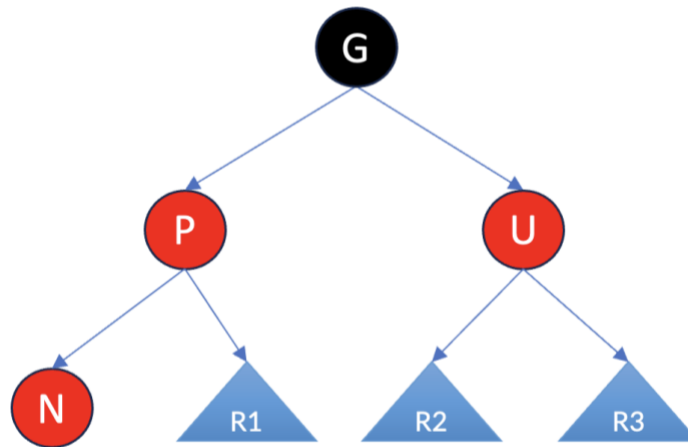
2. Đổi màu G từ đen sang đỏ:

Mục đích là để đảm bảo tổng số node đen trên mọi đường đi từ gốc đến lá không thay đổi (duy trì “chiều cao đen”).

Tuy nhiên, thao tác này có thể dẫn đến việc **G trở thành node đỏ có cha cũng là đỏ**, tạo ra vi phạm mới ở mức cao hơn.

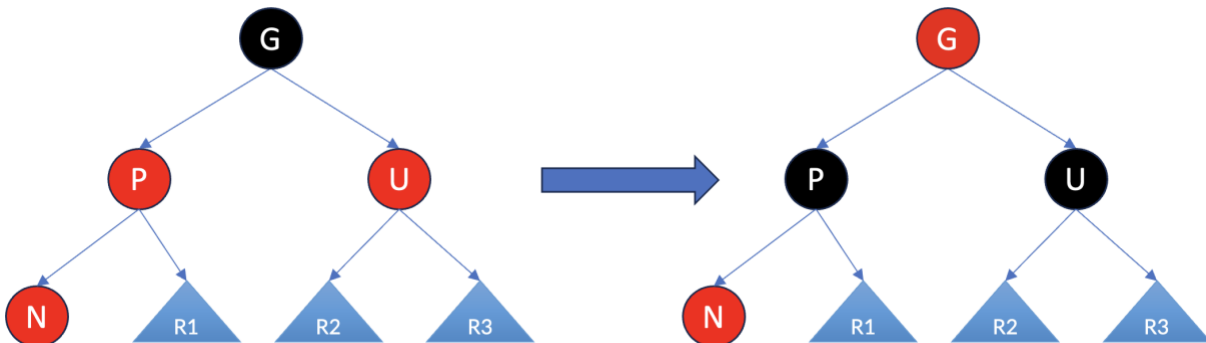
Khi đó, thuật toán sẽ **đệ quy (fix-up) tiếp tục trên G**, coi G như node mới được chèn, và xử lý lại từ đầu các quy tắc.

Ví dụ: Trước khi fix-up:



Hình IV.2.3.1. TH3: trước khi fix-up

Sau khi điều chỉnh sẽ thành:



Hình IV.2.3.2. TH3: sau khi fix-up

Nếu sau khi đổi màu, node G trở thành gốc của cây (root), thì ta thực hiện thêm một bước đặc biệt:

⇒ **Luôn đổi màu G trở lại thành đen**, để thỏa mãn quy tắc “node gốc phải là đen”.

Kết luận: Trường hợp 3 là một trong những bước quan trọng nhất của quá trình chèn trong Red-Black Tree. Nó đảm bảo rằng sự cân bằng màu sắc được duy trì một cách nhẹ nhàng thông qua việc **hoán đổi màu**, thay vì xoay cây phức tạp.

Tuy nhiên, nếu quá trình đổi màu làm xuất hiện vi phạm ở mức cao hơn, thuật toán sẽ tiếp tục xử lý đệ quy lên phía trên cho đến khi cây đạt trạng thái hợp lệ.

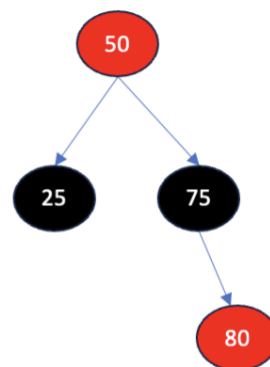
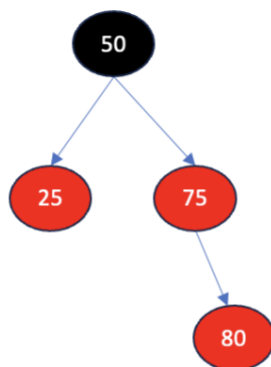
Một ví dụ đơn giản về insertion:

Tạo cây đỏ đen từ dãy số: 50, 75, 25, 80,...

Thêm 50



Thêm 75 25 80



TH3: P và U đều là node đỏ
➤ Đổi 75 và 25 thành đen
➤ Đổi 50 thành đỏ

TH1: Node gốc màu đen
➤ Đổi 50 thành đen

2.4. TH4: P là node đỏ và U là node đen

Khi node mới (N) được chèn vào và tô đỏ, nếu **node cha (P)** của nó cũng **màu đỏ**, trong khi **node chú (U)** là **đen (hoặc không tồn tại)**, thì cây sẽ **vi phạm quy tắc số 4 của cây đỏ-đen**:

⇒ **Quy tắc 4:** Không được tồn tại hai node đỏ kề nhau (một node đỏ không thể có cha là đỏ).

Trường hợp này không thể khắc phục bằng việc **đổi màu** như trong các trường hợp trước, mà yêu cầu **thay đổi cấu trúc cây thông qua phép xoay (rotation)**.

Mục tiêu của xoay là **đưa node cha (P)** hoặc **node mới (N)** lên vị trí cao hơn, đảm bảo cấu trúc cân bằng và khôi phục lại thứ tự tìm kiếm của cây nhị phân.

Phân loại chi tiết các tình huống

Trường hợp này chia thành **bốn kiểu con**, phụ thuộc vào việc node P là con trái hay con phải của node ông (G), và node N nằm bên nào của P.

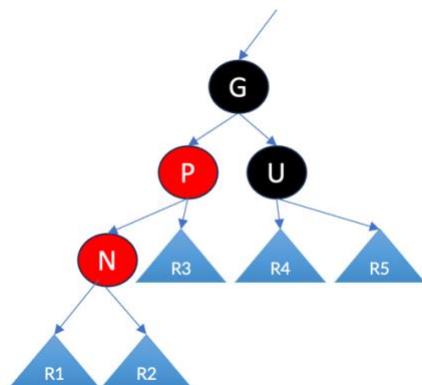
Trường hợp con	Mô tả vị trí	Ký hiệu	Cách xử lý
1. Left-Left (LL)	P là con trái của G, N là con trái của P	(G–P–N đều nằm bên trái)	Thực hiện xoay phải (Right Rotation) quanh G
2. Left-Right	P là con trái của G, N là con phải của P		Thực hiện xoay trái quanh P, sau đó LL
3. Right-Right (RR)	P là con phải của G, N là con phải của P	(G–P–N đều nằm bên phải)	Thực hiện xoay trái (Left Rotation) quanh G
4. Right-Left	P là con phải của G, N là con trái của P		Thực hiện xoay phải quanh P, sau đó RR

2.4.1. P là con trái của G, N là con trái của P (Left – Left)

Mô tả:

Đây là tình huống khi **node cha (P)** là **con trái của node ông (G)** và **node mới chèn (N)** lại là **con trái của P**.

Cấu trúc ban đầu của cây khi đó có dạng:



Hình IV.2.4.1.2. Left-Left

Trường hợp này **vi phạm quy tắc số 4** của cây đỏ-đen:

⇒ Node đỏ không được có cha cũng là node đỏ.

Lý do: N và P đều là đỏ, tạo nên chuỗi hai node đỏ liên tiếp trên cùng đường đi.

Hướng xử lý

Để khắc phục vi phạm và khôi phục cân bằng màu sắc, ta thực hiện **xoay phải (Right Rotation)** quanh node ông (G), kết hợp với việc **đổi màu giữa G và P**.

Các bước cụ thể:

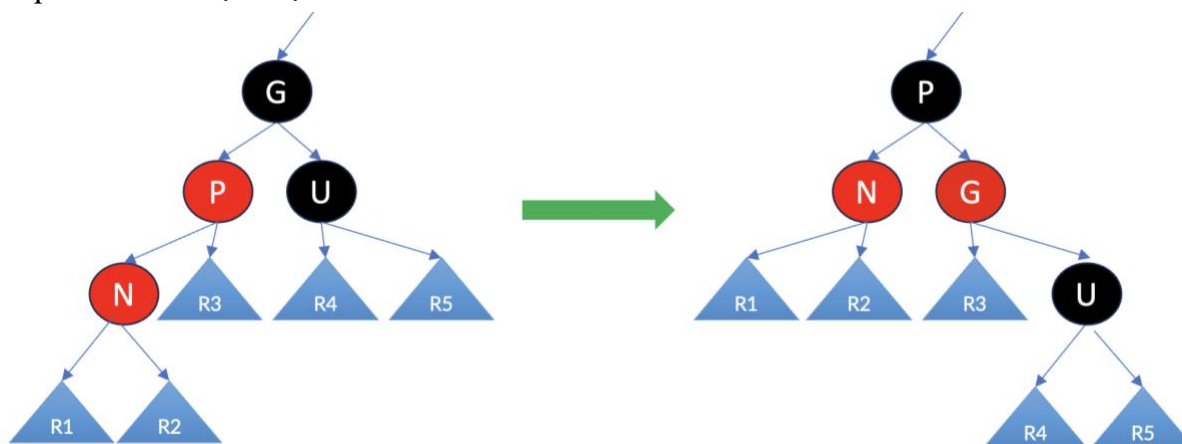
1. **Xoay phải (Right Rotate)** tại node G:

- Node P sẽ trở thành node cha mới.
- Node G trở thành con phải của P.
- Node N giữ nguyên vị trí là con trái của P.

2. **Đổi màu:**

- Node **P** (vừa lên vị trí cao hơn) được đổi sang **đen**.
- Node **G** (bị hạ xuống dưới) được đổi sang **đỏ**.

Kết quả sau khi thực hiện:



Hình IV.2.4.1.2. Left-Left sau fix

Giải thích và phân tích

- Sau khi xoay, cây đã loại bỏ được chuỗi hai node đỏ liên tiếp (N và P).
- Chiều cao đen trên tất cả các đường đi từ gốc đến lá được bảo toàn.
- Cấu trúc cây được giữ đúng dạng **Binary Search Tree (BST)** vì quan hệ giá trị giữa các node vẫn hợp lệ.
- Thao tác xoay chỉ tác động cục bộ (tại G và P), không ảnh hưởng đến các nhánh khác.

Kết quả sau điều chỉnh

- Cây khôi phục được toàn bộ năm tính chất của Red-Black Tree.

- Không cần tiếp tục đệ quy lên các cấp cao hơn, vì vi phạm đã được loại bỏ hoàn toàn tại nút G.
- Đây được xem là **trường hợp cơ bản nhất của phép xoay trong Red-Black Tree**, tương tự với **trường hợp Right-Right (RR)** nhưng đảo hướng.

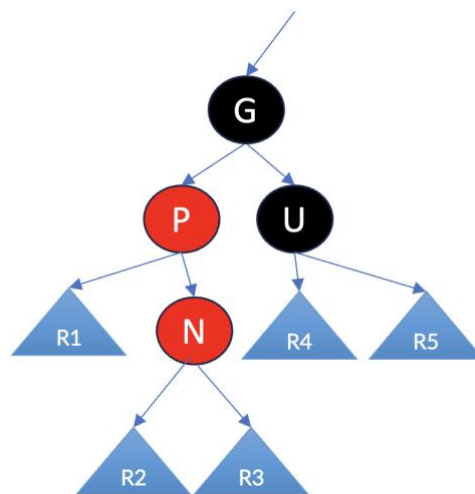
2.4.2. Left – Right: P là con trái của G, N là con phải của P

Mô tả:

Trường hợp **Left-Right** xảy ra khi:

- Node cha (**P**) là **con trái của ông (G)**,
- Node mới được chèn (**N**) là **con phải của P**.

Cấu trúc ban đầu của cây như sau:



Hình IV.2.4.2.1 Left-right

Tình huống này **vi phạm quy tắc 4** của Red-Black Tree:

⇒ Node đỏ không được có cha cũng là node đỏ.

Trong đó, **P** và **N** đều có màu đỏ, dẫn đến chuỗi hai node đỏ liên tiếp.

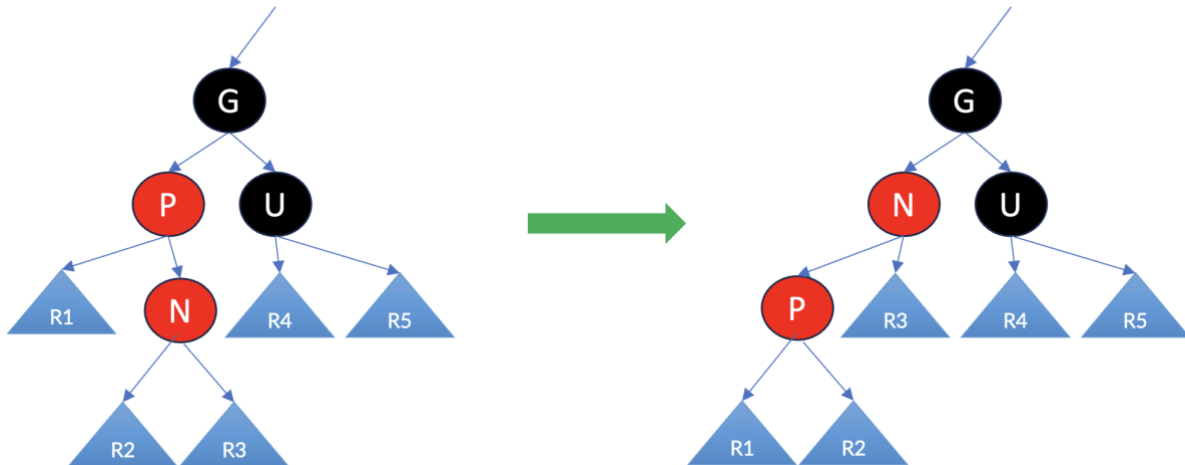
Hướng xử lý

Đối với dạng **Left-Right**, cây không thể khôi phục cân bằng bằng một phép xoay đơn, mà cần **thực hiện hai bước liên tiếp (xoay kép)**:

1. Bước 1 – Xoay trái (Left Rotation) quanh node P:

- Node N sẽ di chuyển lên vị trí của P.
- Node P trở thành **con trái của N**.
- Cấu trúc cây sau bước này chuyển về dạng **Left-Left (LL)** kinh điển.

Kết quả trung gian:

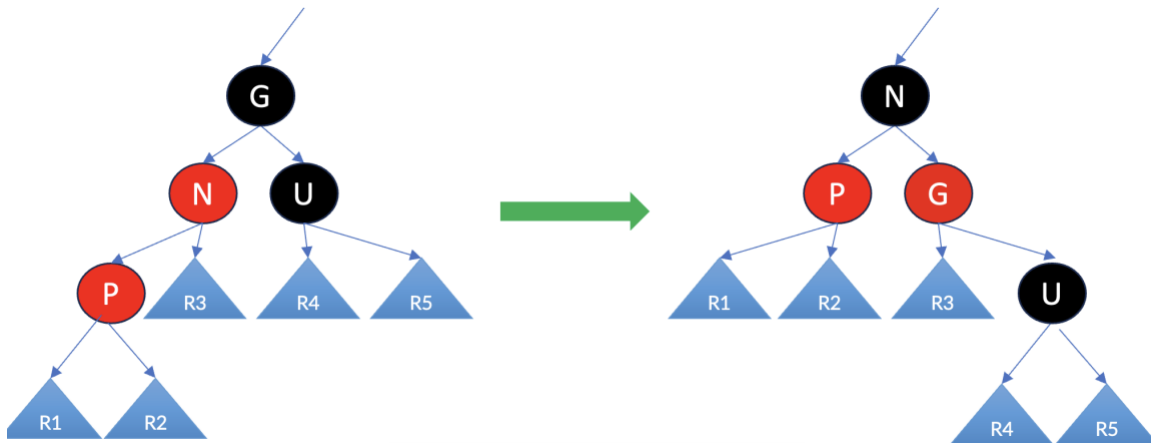


Hình IV.2.4.2.2. Left-right sau fix 1

2. Bước 2 – Xoay phải (Right Rotation) quanh node G:

- Node N trở thành node cha mới của cả G và P.
- Sau xoay, thực hiện **đổi màu**:
 1. Node N → Black
 2. Node G → Red

Kết quả cuối cùng:



Hình IV.2.4.2.3. Left-right sau fix 2

Phân tích sau khi điều chỉnh

- Cây không còn hai node đỏ kề nhau.
- Số node đen trên mọi đường đi từ gốc đến lá vẫn được bảo toàn.
- Cấu trúc nhị phân tìm kiếm (BST) vẫn hợp lệ.
- Cây trở lại trạng thái cân bằng màu sắc và chiều cao đen.

Giải thích trực quan

Ta có thể hiểu **Left-Right** là một trường hợp “gập khúc”:

- Node mới N nằm lệch hướng so với ông G (một bên trái, một bên phải).

- Do đó, cần **xoay một lần để thẳng hàng** ($P \leftrightarrow N \leftrightarrow G$), rồi **xoay tiếp để cân bằng**.

Như vậy, việc xử lý **Left–Right** chỉ là một bước trung gian để đưa cây về dạng **Left–Left** trước khi áp dụng quy trình quen thuộc (xoay phải + đổi màu).

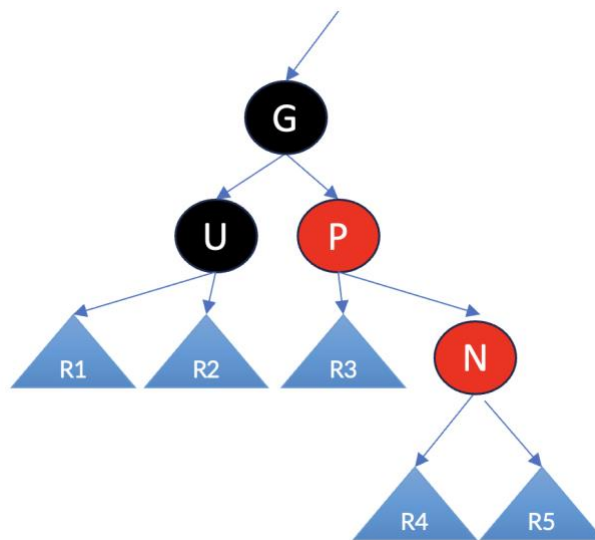
2.4.3. Right – Right: P là con phải của G, N là con phải của P

Mô tả:

Trường hợp **Right–Right (RR)** xảy ra khi:

- Node cha (**P**) là **con phải của node ông (G)**,
- Node mới được chèn (**N**) là **con phải của P**.

Cấu trúc ban đầu của cây như sau:



Hình IV.2.4.3.1.Right-right

Trường hợp này **vi phạm quy tắc số 4** của cây đỏ-đen:

⇒ Node đỏ không được có cha cũng là node đỏ.

Cụ thể, cả **P** và **N** đều có màu đỏ, tạo nên một chuỗi hai node đỏ liên tiếp trên cùng nhánh.

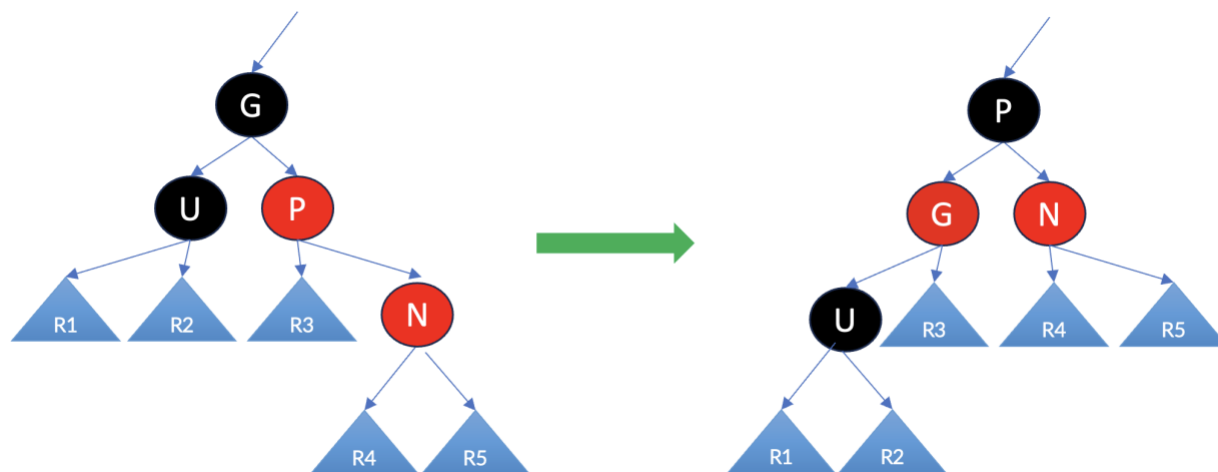
Hướng xử lý

Để khắc phục vi phạm, thuật toán thực hiện các bước sau:

1. **Xoay trái (Left Rotation)** tại node ông **G**:
 - Node **P** được đưa lên làm cha thay thế vị trí của **G**.
 - Node **G** trở thành con trái của **P**.
 - Node **N** vẫn giữ nguyên vị trí là con phải của **P**.
2. **Đổi màu**:
 - Node **P** (vừa được đưa lên) chuyển sang **màu đen**.
 - Node **G** (bị hạ xuống dưới) đổi thành **màu đỏ**.

Kết quả sau khi điều chỉnh

Cấu trúc cây sau khi thực hiện xoay và đổi màu:



Hình IV.2.4.3.2. Right-right sau fix

Phân tích và nhận xét

- Trường hợp Right–Right là **đối xứng của Left–Left**: nếu ta lật gương toàn bộ cây theo trục dọc, hai quy trình này là tương đương nhau.
- Cả hai đều được xử lý bằng **một phép xoay đơn (single rotation)** và **hoán đổi màu giữa G và P**.
- Đây là thao tác cơ bản nhất giúp cây tự điều chỉnh khi mất cân bằng ở phía bên phải.

Kết luận

Sau khi thực hiện phép xoay và đổi màu:

- Cây loại bỏ được chuỗi node đỏ liên tiếp,
- Cấu trúc BST vẫn được duy trì,
- Chiều cao đen của cây không thay đổi,
- Red-Black Tree trở lại trạng thái cân bằng và hợp lệ.

Đây là **một trong hai trường hợp xoay đơn** phổ biến nhất trong Red-Black Tree, đảm bảo việc chèn node mới luôn đạt độ phức tạp $O(\log n)$.

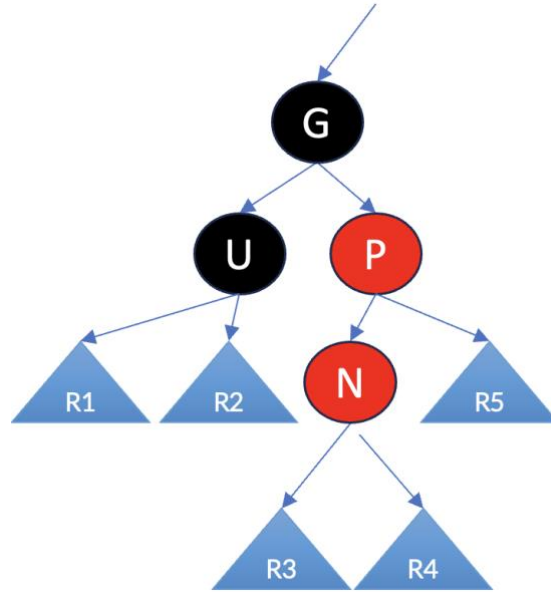
2.4.4. Right – Left: P là con phải của G, N là con trái của P

Mô tả:

Trường hợp **Right–Left (RL)** xảy ra khi:

- Node cha (**P**) là **con phải của ông (G)**,
- Node mới được chèn (**N**) là **con trái của P**.

Cấu trúc ban đầu của cây như sau:



Hình IV.2.4.4.1. Right-right

Cây này **vi phạm quy tắc số 4** của Red-Black Tree:

⇒ Node đỏ không được có cha cũng là node đỏ.

Cụ thể, N và P đều có màu đỏ, tạo thành chuỗi hai node đỏ liên tiếp.

Hướng xử lý

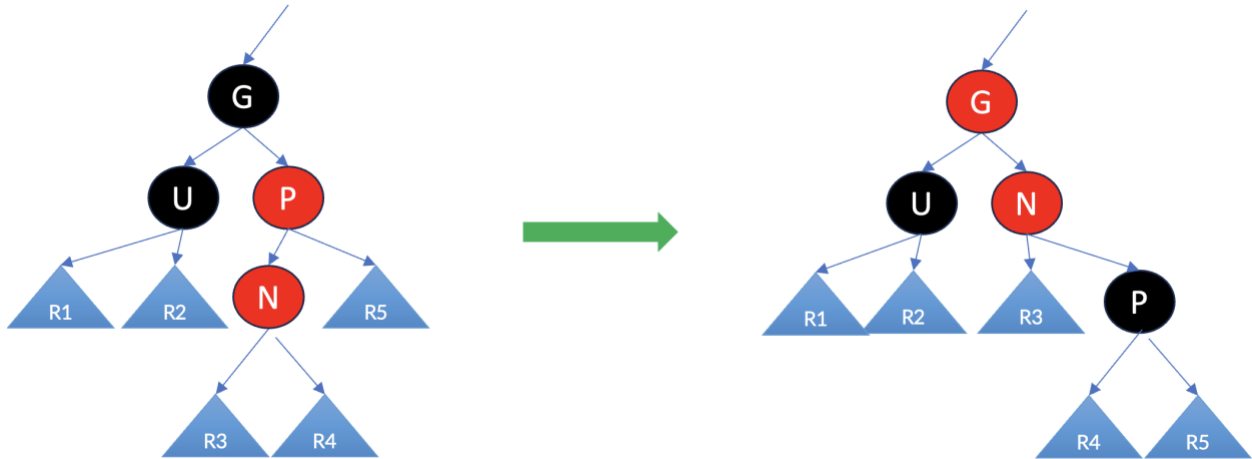
Trường hợp Right–Left không thể khắc phục bằng một phép xoay đơn vì hình dạng cây bị “gập khúc” (lệch hướng giữa G, P và N).

Do đó, thuật toán phải **thực hiện hai phép xoay liên tiếp (double rotation)** để khôi phục cân bằng:

1. Bước 1 – Xoay phải (Right Rotation) quanh node P:

- Node N di chuyển lên vị trí của P.
- Node P trở thành con phải của N.
- Sau khi xoay, cấu trúc cây tạm thời chuyển về dạng **Right–Right (RR)**.

Cấu trúc trung gian:



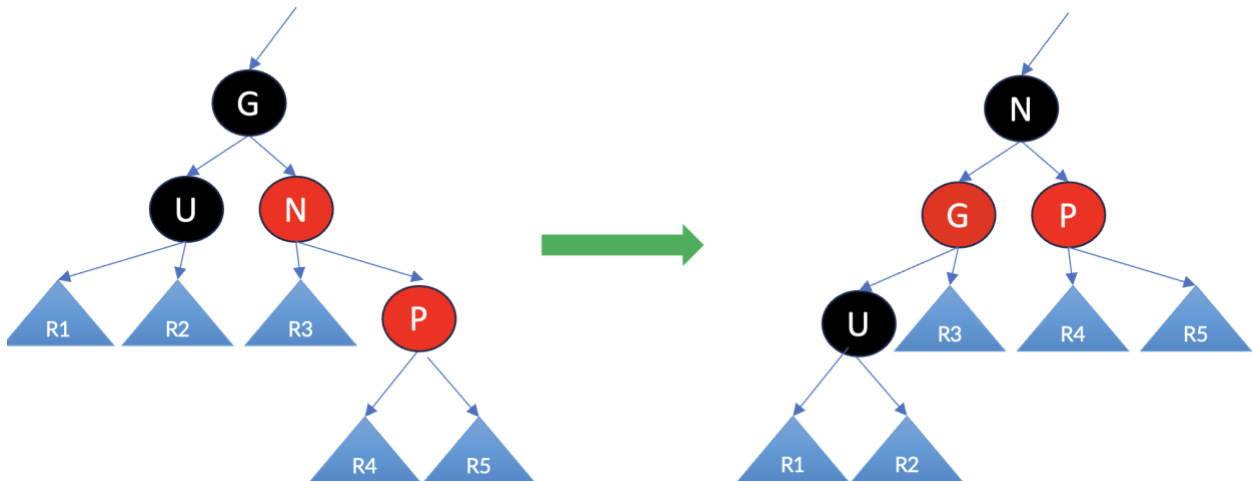
Hình IV.2.4.4.2.right-left sau fix 1

2. Bước 2 – Xoay trái (Left Rotation) quanh node G:

- Node N được đưa lên vị trí của G.
- Node G trở thành con trái của N, P giữ vai trò con phải của N.
- Sau xoay, đổi màu để khôi phục quy tắc màu sắc:

1. Node N → Black
2. Node G → Red

Kết quả cuối cùng:



Hình IV.2.4.4.3.right-left sau fix 2

Phân tích sau khi hiệu chỉnh

- Hai node đỏ kề nhau (P và N) đã được tách rời, đảm bảo không còn vi phạm quy tắc.
- Số node đen trên mọi đường đi từ gốc đến lá được bảo toàn (chiều cao đen không thay đổi).
- Cây giữ nguyên tính chất tìm kiếm nhị phân (BST property).
- Cấu trúc cây trở lại trạng thái cân bằng cả về **hình dạng** lẫn **màu sắc**.

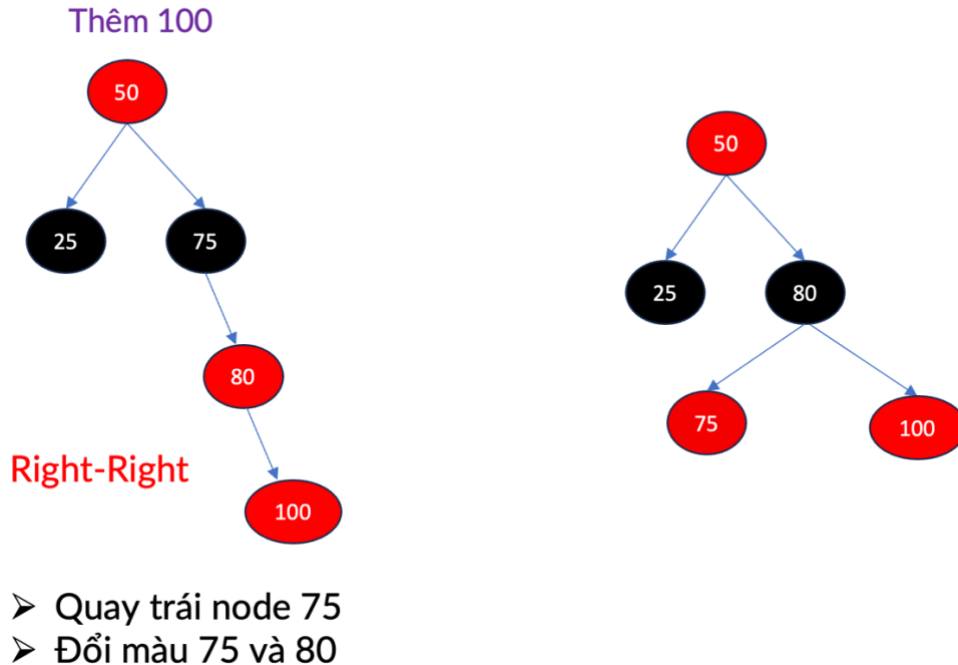
Giải thích trực quan

Trường hợp Right–Left là **phiên bản phản chiếu của Left–Right**:

- Ở Left–Right, ta xoay trái rồi xoay phải.
- Ở Right–Left, ta xoay phải rồi xoay trái.

Hai thao tác này đều nhằm “đuỗi thẳng” nhánh gấp khúc và đưa node trung gian (N) lên giữa để cân bằng lại cây.

Ví dụ: Tạo cây đỏ đen từ dãy số: 50, 75, 25, 80, 100,...



3. DELETE (Xoá)

Thao tác **xóa một node** trong cây đỏ-đen là một trong những bước phức tạp nhất, bởi nó không chỉ cần xóa node khỏi cấu trúc cây nhị phân tìm kiếm (BST) mà còn phải **duy trì năm tính chất của cây đỏ-đen** sau khi xóa.

Việc mất đi một node, đặc biệt là node **đen**, có thể làm **thay đổi số lượng node đen trên một số đường đi**, dẫn đến **mất cân bằng “chiều cao đen”** — đây là vấn đề chính mà thuật toán cần khắc phục.

Quy trình tổng quát:

Việc xóa một node X khỏi cây đỏ-đen được thực hiện qua **ba giai đoạn chính**:

1. **Tìm node cần xóa:**

Thực hiện như trong cây nhị phân tìm kiếm thông thường.

- Nếu **không tìm thấy node X**, thuật toán dừng lại.
- Nếu **tìm thấy**, ta tiếp tục bước 2.

2. Tiến hành xóa:

Tùy thuộc vào cấu trúc của node cần xóa, chia thành ba trường hợp:

- **Trường hợp 1:** Node cần xóa là lá (không có con).
- **Trường hợp 2:** Node cần xóa có đúng 1 con.
- **Trường hợp 3:** Node cần xóa có 2 con → ta tìm node thay thế (successor hoặc predecessor) để hoán đổi giá trị, rồi quy bài toán về trường hợp 1 hoặc 2.

3. Hiệu chỉnh sau khi xóa (Fix-up):

Sau khi xóa, nếu việc loại bỏ node làm vi phạm các tính chất đỏ–đen (đặc biệt là chiều cao đen), thuật toán sẽ thực hiện các phép đổi màu và xoay cây để khôi phục cân bằng.

Phân loại theo số con của node bị xóa:

1. Node cần xóa là lá (không có con)

- Nếu node là **đỏ** → có thể **xóa trực tiếp** mà không ảnh hưởng đến cây.
- Nếu node là **đen** → việc xóa làm giảm chiều cao đen trên đường đi → cần xử lý bổ sung (Fix Double Black).

2. Node cần xóa có 1 con

- Nếu node bị xóa là **đen**, con của nó **phải là đỏ** (theo tính chất đỏ–đen). Khi đó, ta có thể **thay thế node bị xóa bằng con của nó và đổi màu con thành đen**, đảm bảo cân bằng màu sắc.
- Nếu node bị xóa là **đỏ**, ta xóa trực tiếp mà không ảnh hưởng tới các tính chất.

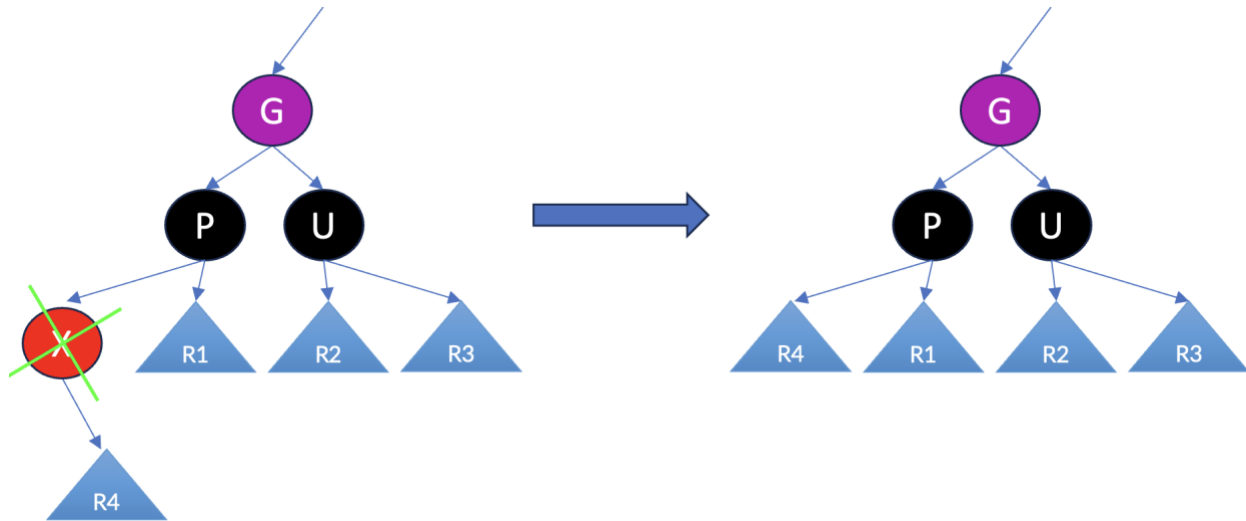
3. Node cần xóa có 2 con

- Ta tìm **node kế tiếp (successor)** hoặc **node liền trước (predecessor)** — đây là node nhỏ nhất trong cây con phải (hoặc lớn nhất trong cây con trái).
- Hoán đổi giá trị giữa node cần xóa và node thay thế.
- Sau đó, tiếp tục thực hiện xóa trên node thay thế (lúc này sẽ rơi vào trường hợp 1 hoặc 2).

3.1. TH1: X màu đỏ hoặc nút gốc

- Khi một node **đỏ** bị xóa, số lượng node đen trên các đường đi từ gốc đến lá **không bị thay đổi** (vì node đỏ không được tính vào “chiều cao đen”).
- Ngoài ra, node đỏ luôn có **cha là đen** theo quy tắc đỏ–đen, nên việc xóa node đỏ **không làm mất tính hợp lệ của cấu trúc**.

Ví dụ:



Hình IV.3.1. Delete 1

Kết quả:

- Cây không thay đổi “chiều cao đen”.
- Không tồn tại hai node đỏ kề nhau.
- Không cần thao tác xoay hoặc đổi màu.

Hệ quả

Trong cả hai trường hợp:

- **Không vi phạm tính chất đỏ–đen**, vì không xuất hiện chuỗi đỏ–đỏ hoặc thay đổi số lượng node đen trên bất kỳ đường nào.
- **Không cần điều chỉnh (fix-up)**, cây vẫn hợp lệ sau thao tác xóa.

Kết luận

Trường hợp 1 là tình huống **đơn giản nhất** trong quá trình xóa node khỏi Red-Black Tree:

- Nếu node bị xóa là **đỏ** → **xóa trực tiếp**.
- Nếu node bị xóa là **gốc** → **xóa và tô đen lại root mới (nếu có)**.

Không cần thực hiện bất kỳ phép xoay hay đổi màu nào.

Cây sau khi xóa vẫn đảm bảo đầy đủ **5 tính chất đỏ–đen**, duy trì độ phức tạp trung bình **$O(\log n)$** cho thao tác.

3.2. TH2: X là node đen và có 1 con là nút đỏ

Mô tả

- Khi node cần xóa (X) là **đen**, và nó có **chính xác một node con** (trái hoặc phải), **node con này là đỏ**, thì việc xóa node X sẽ làm **mất đi một node đen** trên đường đi từ gốc đến các lá.
- Tuy nhiên, do **node con của X là đỏ**, ta có thể **điều chỉnh màu** của node con để **bù lại phần “đen” bị mất**.

Phân tích nguyên nhân:

Theo định nghĩa của Red-Black Tree:

- Mỗi đường đi từ gốc đến lá phải có **cùng số lượng node đen**.
- Khi một node đen bị xóa, chiều cao đen của cây giảm trên đường đi chứa node đó.

Nếu node X có một con đỏ, thì:

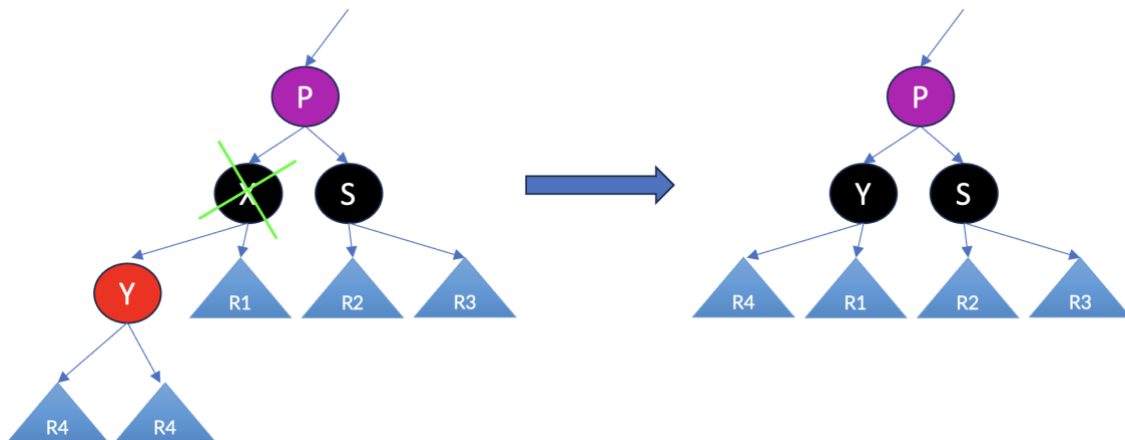
- Node con đó có thể **đóng vai trò thay thế (successor)** cho node X.
- Sau khi xóa X, ta **đưa node con đỏ lên vị trí của X** và **đổi màu node này thành đen**.

Nhờ vậy:

- Số lượng node đen trên mỗi đường đi được **bảo toàn**.
- Không cần thực hiện các phép xoay phức tạp.

Bước xử lý:

1. Xóa node X khỏi cây.
2. Đưa node con Y lên thay thế vị trí của X.
3. Đổi màu của node Y thành **đen**.



Hình IV.3.2. Delete 2

Giải thích:

- Việc đổi màu của node Y từ đỏ sang đen giúp **duy trì chiều cao đen** của cây.
- Không xuất hiện hai node đỏ liên tiếp, vì node cha (P) có thể đỏ hoặc đen đều hợp lệ.
- Không cần thực hiện xoay, vì hình dạng cây vẫn duy trì tính chất BST.

Kết quả và hệ quả:

Sau khi thực hiện:

- Cây tiếp tục **đảm bảo 5 tính chất đỏ–đen**.
- Không xảy ra tình trạng “Double Black” (thừa đen).
- Toàn bộ thao tác chỉ cần một lần đổi màu, không cần xoay.

3.3. TH3: X là node đen và có 2 con là nút đen

Mô tả hiện tượng

Khi node cần xóa (X) là **đen**, và node này có **hai con đen (hoặc NIL)**, thì việc xóa X dẫn đến **mất một node đen trên đường đi** từ gốc đến các node lá.

Hậu quả là cây **vi phạm tính chất số 5** của cây đỏ–đen:

⇒ “Mọi đường đi từ một node đến các node lá NIL của nó đều chứa cùng số lượng node đen.”

Tình trạng này được gọi là **Double Black** – tức là, tại vị trí thay thế node X, ta “mang theo một màu đen thừa” cần xử lý để khôi phục cân bằng.

Phân tích cấu trúc

Để giải quyết vấn đề, ta gọi **S là node anh em (Sibling)** của X.

Mục tiêu của quá trình hiệu chỉnh là **loại bỏ màu đen dư thừa** tại vị trí của X hoặc di chuyển nó lên phía trên cây.

Có **4 trường hợp chính** tùy theo màu của S và màu của các node con của S:

1. **S màu đen và cả hai con của S đều đen**
→ Thực hiện **đổi màu S** sang đỏ và **chuyển Double Black lên cha P**.
2. **S màu đen và con trái của S màu đỏ**
→ Thực hiện **xoay phải tại S**, sau đó xử lý tương tự trường hợp 3.
3. **S màu đen và con phải của S màu đỏ**
→ Thực hiện **xoay trái tại P**, đồng thời đổi màu giữa P và S.
4. **S màu đỏ**
→ Thực hiện **xoay trái hoặc phải (tùy hướng)** để biến thành một trong ba trường hợp trên.

3.3.1. TH1: X là node đen, X có 2 con là node đen, và S màu đen (cả 2 con của S cũng đen)

Trong quá trình xóa, đây là **trường hợp đơn giản nhất** trong nhóm “Double Black Case”. Sau khi node X bị xóa, vị trí X để lại một **Double Black**. Khi xem xét node anh em (**Sibling**) của X – ký hiệu là S, ta nhận thấy:

- **S có màu đen**
- **Hai node con của S cũng đều màu đen (hoặc NIL đen)**
- Node cha **P** có thể màu đen hoặc đỏ, nhưng không ảnh hưởng đến việc nhận diện trường hợp này.

Nhận diện vấn đề: xuất hiện Double Black

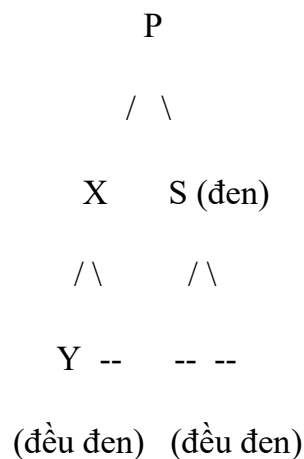
Khi X là node **đen**, việc xóa X gây ra việc **thiếu một node đen** trên các đường đi qua nhánh của X:

Đường đi qua X thiếu 1 node đen

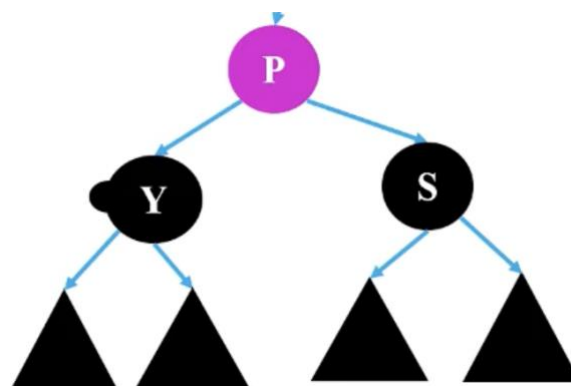
⇒ Vi phạm tính chất số 5 của cây đỏ đen.

Để khôi phục cân bằng, ta phải tìm cách **đẩy màu đen lên trên** hoặc **tái phân phối màu đen** thông qua các phép đổi màu.

Trước khi xử lý, cây có dạng:



- X bị xóa → để lại **Double Black** ở vị trí Y/NIL.
- S là node đen, và hai con của S đều đen → đủ điều kiện của **Case 1**.



Hình IV.3.3. Delete 3

Cách xử lý

Vì S và cả hai con của nó đều đen, cây cho phép “mượn” một màu đen từ S để bù vào phần thiếu hụt ở vị trí Double Black.

Thao tác như sau:

Bước 1 – Đổi màu S thành đỏ

Mục tiêu của bước này:

- Giảm số node đen ở nhánh bên S đi **1**, giúp hai nhánh từ P xuống Y và S trở nên cân bằng lại.
- Đồng thời chuyển “trách nhiệm xử lý Double Black” lên node P.

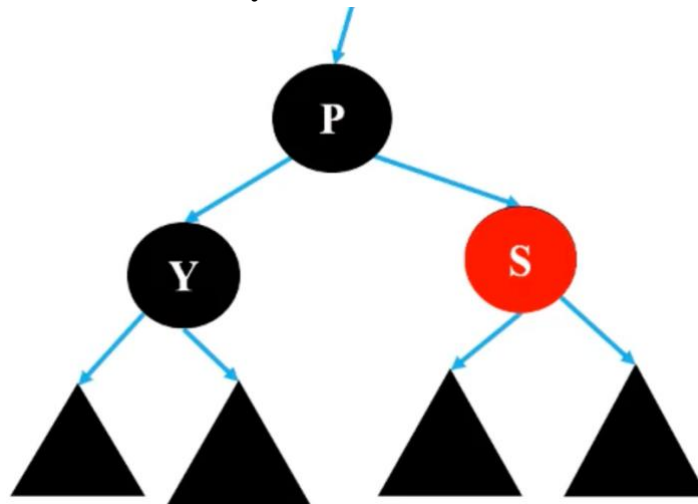
Bước 2 – Xóa trạng thái Double Black tại X/Y

Xóa Double Black tại vị trí con của X, vì đã được bù trừ.

Bước 3 – Kiểm tra lại P

- Nếu P **đỏ**, ta đổi P thành **đen** và kết thúc:
→ Màu đen được “thêm lại” từ chính node cha P.
- Nếu P **đen**, P trở thành node mang Double Black mới
→ Ta tiếp tục xử lý đệ quy lên phía trên.

Vậy ta sẽ được cây như hình dưới đây:

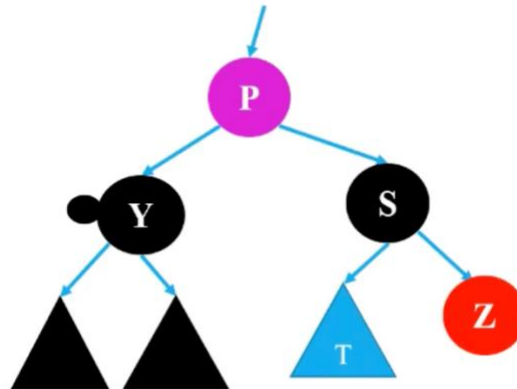


Hình IV.3.3. Delete 4

Trường hợp này là trường hợp nhẹ nhất và đơn giản nhất để xử lý trong thuật toán delete của Red-Black Tree:

- Không cần xoay
- Chỉ thực hiện đổi màu
- Có thể kết thúc ngay hoặc lan truyền lên cha P tùy màu của P

3.3.2. TH2: X là node đen, hai con của X là node đen, S là node đen và con phải của S là màu đỏ



Hình IV.3.3. Delete 5

Trong trường hợp này:

- Node cần sửa màu (Double Black) là **X**
- **S** là node anh em của **X** và có màu **đen**
- Con phải của **S** (**Z**) có màu **đỏ**
- Con trái của **S** (**T**) có thể là **đen hoặc NIL**
- Node cha **P** (màu tím trong slide) có thể là **đỏ hoặc đen**

Đây được xem là **trường hợp “Right – Red Child”** của **S**.

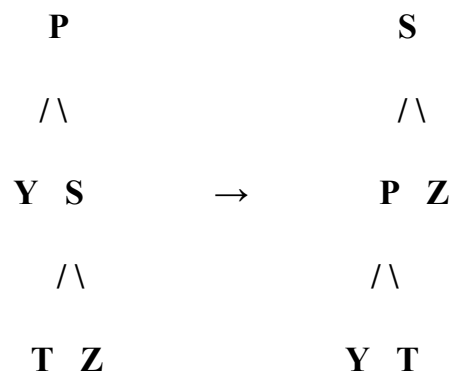
Mục tiêu của xử lý TH2

- Loại bỏ trạng thái **Double Black** tại **X**
- Giữ nguyên tính chất:
 - Không có hai node **đỏ** liên tiếp
 - Số lượng node **đen** trên mọi đường từ gốc đến lá phải bằng nhau

Các bước xử lý:

1. Quay trái tại **P**

Ta thực hiện phép quay trái quanh node **P**:



- P trở thành con trái của S
- T được chuyển thành con phải của P
- Cấu trúc cây dần trở về dạng cân bằng

2. Đảo màu P và S

Sau quay trái, để khôi phục tính chất màu:

- Đổi màu **P** thành **đen**
- Đổi màu **S** thành **đỏ**

Mục đích:

- ✓ Giữ số node đen trên mỗi đường đi không đổi
- ✓ Tránh vi phạm “cha đỏ – con đỏ”

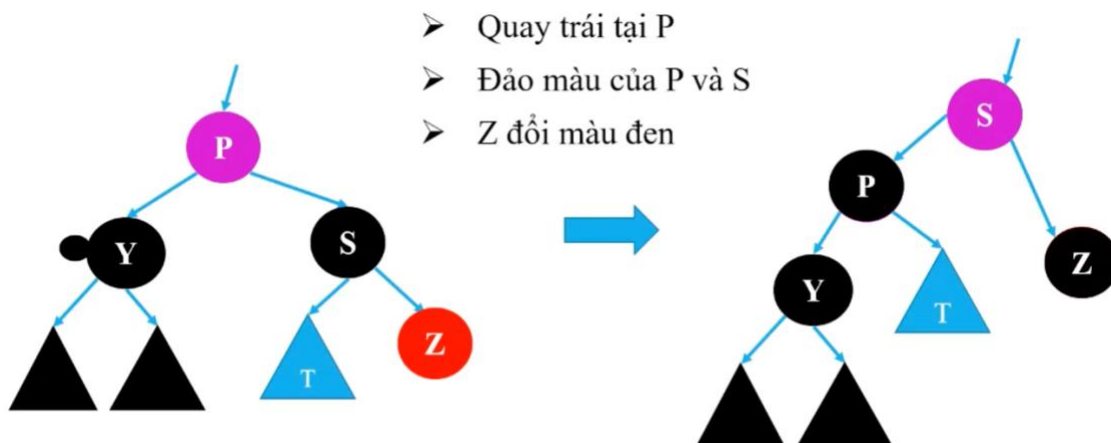
3. Z đổi sang màu đen

Vì Z (con phải của S) là node đỏ duy nhất giúp cân bằng cây trong trường hợp này, nên sau quay và đổi màu:

- **Z được đổi sang màu đen**

→ Điều này xóa bỏ Double Black tại X.

Kết quả cuối cùng:

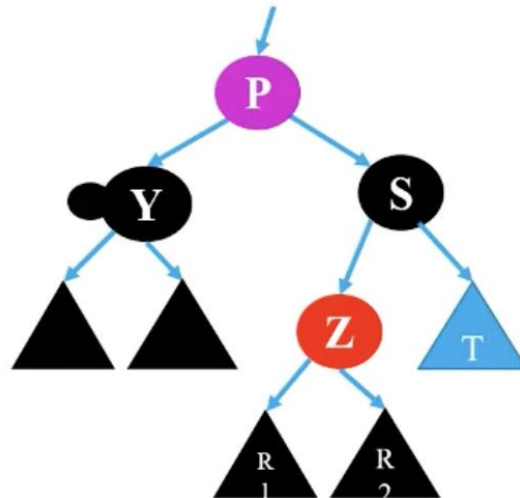


Hình IV.3.3. Delete 6

Sau 3 bước:

- Double Black tại X được xử lý hoàn toàn
- Cây giữ nguyên tính chất đỏ–đen
- Không cần xử lý tiếp tục lên cao hơn

3.3.3. X là node đen, X có 2 con là node đen, S màu đen và con trái của S là màu đỏ, con phải của S là màu đen.



Hình IV.3.3. Delete 7

Đây là trường hợp cân bằng “ngiên trái”, trong đó node anh em S có cấu hình đặc biệt:

- S là node đen
- **Con trái** của S (Z) có màu **đỏ**
- **Con phải** của S (T) lại **đen hoặc NIL**
- Node cha P có thể đỏ hoặc đen
- Node X đang là Double Black (do vừa xóa)

Cấu hình này là dạng chưa thể xử lý trực tiếp, nhưng sau một phép quay, ta sẽ chuyển nó về TH2 – là trường hợp đã giải quyết được.

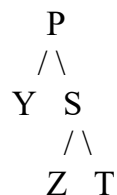
Mục tiêu của việc xử lý TH3:

- Chuyển TH3 → TH2 (trường hợp dễ xử lý)
- Giữ nguyên số node đen trên mỗi đường từ gốc đến lá
- Tránh vi phạm quy tắc “cha đỏ – con đỏ”

Các bước xử lý:

1. Quay phải tại S:

Cấu trúc ban đầu:



Z nằm bên trái của S và là node **đỏ**, nên ta quay **phải tại S**:



→ Z được đưa lên cao hơn để chuẩn bị chuyển sang TH2.

2. Đổi màu Z và S

Sau quay, để tránh tạo vi phạm:

- Z đổi sang **màu đen**
- S đổi sang **màu đỏ**

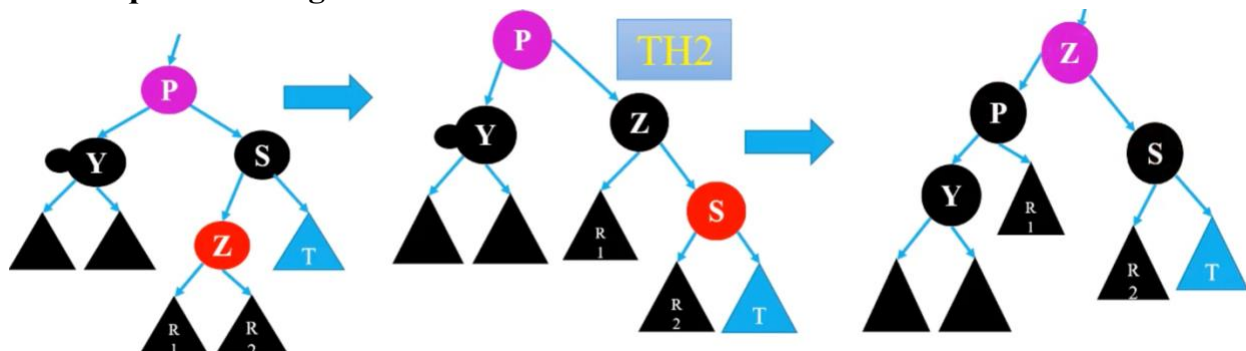
Tại thời điểm này, cây trở về đúng cấu hình của **TH2**.

3. Xử lý tiếp như TH2

Cụ thể:

- S lúc này đóng vai trò giống Z trong TH2
- Tiếp tục thực hiện:
 - ✓ Quay trái tại P
 - ✓ Đổi màu phù hợp ($P \leftrightarrow S$)
 - ✓ Xoá trạng thái Double Black

Kết quả cuối cùng:



Hình IV.3.3. Delete 8

- Double Black được xử lý hoàn toàn

- Không còn vi phạm màu sắc
- Cây trở về trạng thái cân bằng của Red-Black Tree

3.3.4. TH4: X là node đen, X có 2 con là node đen, S màu đỏ

Trong trường hợp này:

- Node X đang mang trạng thái **Double Black**
- Node anh em S của X có **màu đỏ**
- Cả hai con của X đều đen
- Hai con của S (**R1** và **R2**) đều là node đen (hoặc NIL)

Đây là một dạng **trường hợp đặc biệt** khi sửa cây sau xoá, bởi vì:

👉 Nếu anh em S màu **đỏ**, thì cha P **chắc chắn màu đen**

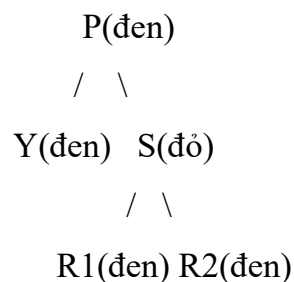
👉 Đây là cấu hình tạm thời gây mất cân bằng, nhưng ta có thể **biến nó thành một trường hợp khác để xử lý hơn**

Mục tiêu của việc xử lý TH4:

- Đưa node S màu đỏ trở về vị trí phù hợp (đặt nó “lên vai” của P)
- Đổi màu để loại bỏ xung đột màu sắc
- Chuyển TH4 → một trong các TH1, TH2 hoặc TH3, khi đó ta xử lý tiếp được

Cách xử lý:

Cấu trúc ban đầu:

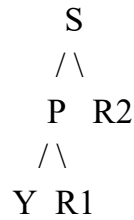


Ở đây:

- X là con trái của Y (có thể coi X là NIL nhưng mang dấu hiệu Double Black)
- S là anh em của X và **S màu đỏ** → đây chính là điều gây vi phạm

Bước 1 — Quay trái tại P

Ta thực hiện phép quay trái tại P để đưa S lên trên, P xuống dưới:



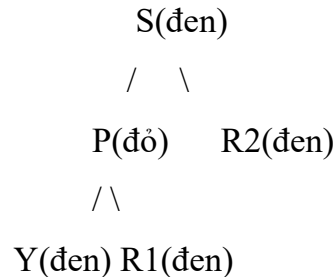
Hình dạng cây trở nên hợp lý hơn để chỉnh màu sắc.

Bước 2 — Đổi màu P thành đỏ, S thành đen

Để đảm bảo không vi phạm quy tắc **cha đỏ – con đỏ**, ta làm:

- S (đang ở trên) $\rightarrow$ đổi sang **đen**
- $P \rightarrow$ đổi sang **đỏ**

Kết quả sau bước này:



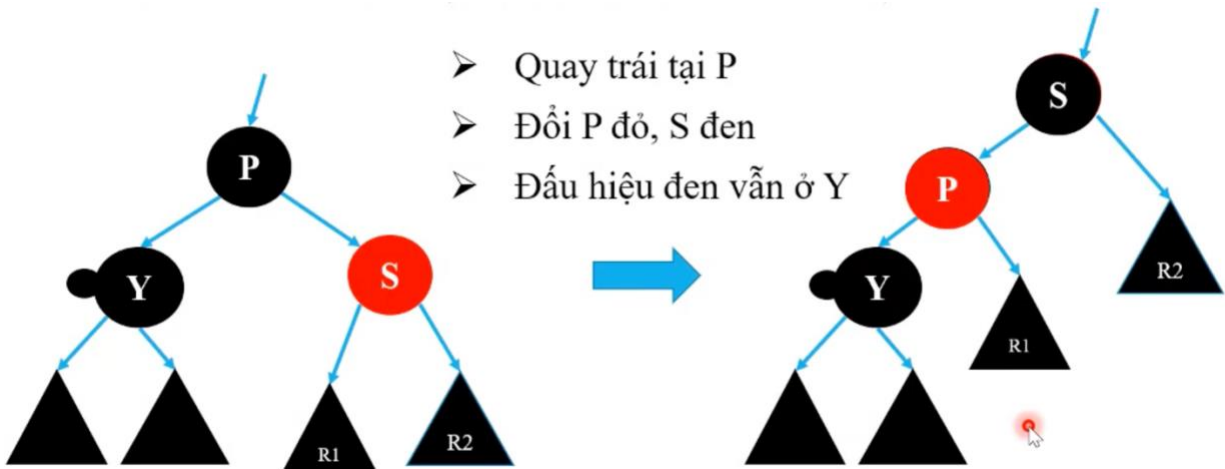
Bước 3 — Trạng thái Double Black vẫn nằm tại Y

Điều quan trọng:

- ✓ Dấu hiệu Double Black **không biến mất**,
- ✓ nhưng **nó được “di chuyển”** sang nhánh Y .

$\Rightarrow$ Như vậy, sau khi thực hiện TH4, ta sẽ chuyển toàn bộ cây vào **một trong các TH1 – TH3**, với một cấu hình dễ xử lý hơn.

Ta được:

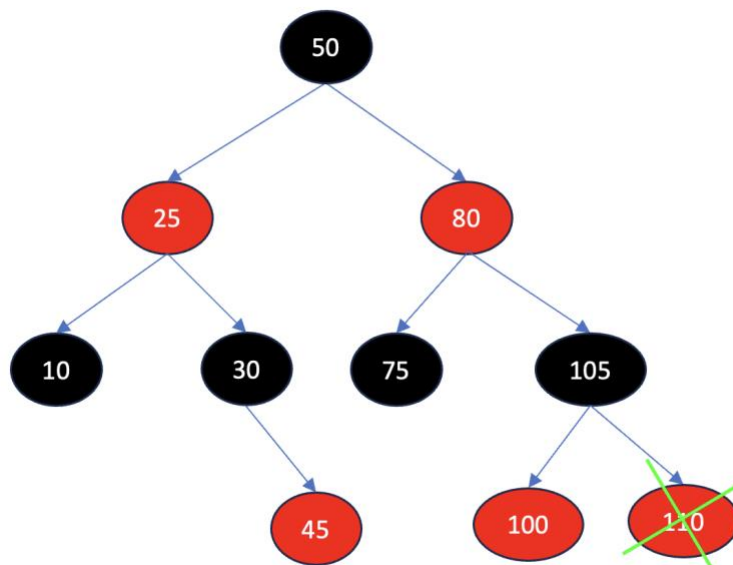


Hình IV.3.3. Delete 9

***Ví dụ về Delete:**

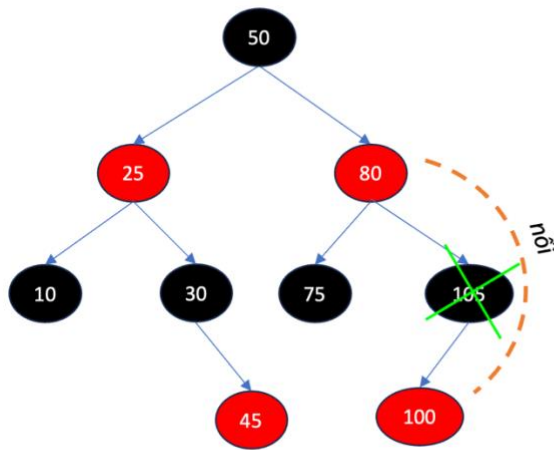
Xoá node 110, 115

Xoá 110

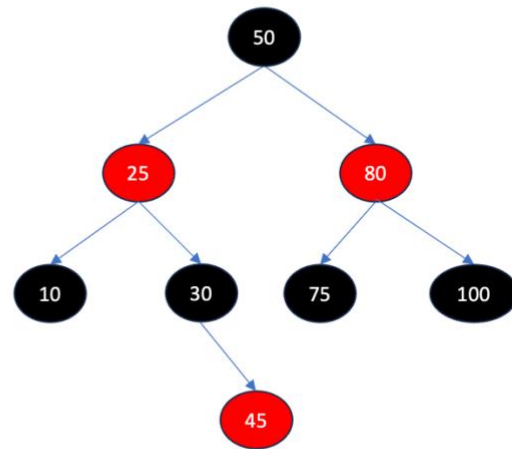


Không ảnh hưởng

Xoá 110



Xoá 105



-> Đổi 100 từ đỏ thành đen

V. ỨNG DỤNG THỰC TIỄN

Cây Đỏ-Đen là một trong những cấu trúc dữ liệu cân bằng nổi tiếng và được sử dụng rộng rãi trong hệ thống hiện đại nhờ khả năng đảm bảo thời gian xử lý $O(\log n)$ cho hầu hết các thao tác quan trọng như chèn, xoá và tìm kiếm. Với tính ổn định, độ cao bị giới hạn chặt chẽ và chi phí duy trì cân bằng thấp, Cây Đỏ-Đen trở thành lựa chọn mặc định trong nhiều thư viện và nền tảng phần mềm.

Dưới đây là các ứng dụng thực tế tiêu biểu:

1. Cấu Trúc Dữ Liệu Trong Thư Viện Lập Trình Chuẩn (Standard Libraries)

Trong hầu hết các ngôn ngữ lập trình hiện đại, Red-Black Tree là một trong những cấu trúc dữ liệu được sử dụng làm **xương sống** cho nhiều kiểu dữ liệu trừu tượng quan trọng. Lý do là vì cây đỏ-đen đảm bảo hiệu năng ổn định $O(\log n)$ cho thao tác tìm kiếm, chèn và xoá – phù hợp với các cấu trúc cần sắp xếp động theo khóa.

1.1. C++ Standard Template Library (STL)

Trong C++ STL, các cấu trúc dữ liệu sau **đều được cài đặt bằng Red-Black Tree**:

- `std::map` — ánh xạ khóa–giá trị được sắp xếp
- `std::set` — tập hợp không trùng lặp, sắp xếp theo thứ tự
- `std::multimap` — ánh xạ khóa–giá trị cho phép khóa trùng
- `std::multiset` — tập hợp cho phép phần tử trùng nhau

Ví dụ:

```

#include <iostream>
#include <map>
using namespace std;
int main() {
    map<int, string> students;    // std::map được cài đặt bằng
    cây đỏ-đen
    students[1] = "An";
    students[3] = "Bình";
    students[2] = "Cường";
    for (auto s : students)
        cout << s.first << " - " << s.second << endl;
    return 0;
}

```

Giải thích hoạt động:

- `std::map<int, string>` tạo ra một cấu trúc ánh xạ có khóa kiểu `int` và giá trị kiểu `string`.
- Khi chèn các phần tử 1, 3, 2, cây đỏ-đen **tự động cân bằng và sắp xếp** để duy trì thứ tự khóa.
- Vòng lặp `for (auto s : students)` duyệt các phần tử theo thứ tự **tăng dần của khóa** — nhờ thuộc tính sắp xếp của cây đỏ-đen.

1.2. Java Collections Framework

Trong **Java Collections Framework**, một số cấu trúc dữ liệu quan trọng như **TreeMap** và **TreeSet** được cài đặt dựa trên **cây Đỏ-Đen (Red-Black Tree)**. Nhờ sử dụng cấu trúc này, các thao tác thêm, xóa và tìm kiếm đều đạt hiệu năng **$O(\log n)$** và ổn định trong mọi trường hợp.

1.2.1. TreeMap

`TreeMap<K, V>` lưu trữ dữ liệu dưới dạng cặp **key – value**, tương tự `HashMap`, nhưng **các phần tử luôn được sắp xếp theo khóa** (ordering).

Nhờ sử dụng cây Đỏ-Đen, `TreeMap`:

- Tự động duy trì trạng thái cân bằng của cây.
- Đảm bảo mọi thao tác:
insert, delete, search → **$O(\log n)$** .

Ví dụ:

```

import java.util.TreeMap;

public class Demo {
    public static void main(String[] args) {
        TreeMap<Integer, String> students = new TreeMap<>();
    }
}

```

```

        students.put(3, "Bình");
        students.put(1, "An");
        students.put(2, "Cường");

        for (var s : students.entrySet()) {
            System.out.println(s.getKey() + " - " +
s.getValue());
        }
    }
}

```

Giải thích:

- TreeMap duy trì thứ tự tự nhiên của khóa (**1, 2, 3**).
- Các thao tác `put()`, `remove()`, `get()` đều là **$O(\log n)$** vì cây luôn được cân bằng theo quy tắc của cây đỏ-đen.
- Thích hợp cho những ứng dụng cần dữ liệu có **thứ tự** hoặc **range search** (tìm theo khoảng giá trị).

1.2.2. TreeSet

`TreeSet<E>` lưu trữ tập hợp không trùng lặp (unique elements) và **luôn được sắp xếp**. Nó sử dụng **TreeMap** bên dưới → đồng nghĩa toàn bộ hoạt động dựa trên **cây Đỏ-Đen**.

Ví dụ:

```

import java.util.TreeSet;

public class Demo {
    public static void main(String[] args) {
        TreeSet<String> names = new TreeSet<>();

        names.add("Cường");
        names.add("An");
        names.add("Bình");

        for (String name : names) {
            System.out.println(name);
        }
    }
}

```

Giải thích:

- TreeSet sắp xếp phần tử theo thứ tự từ điển: **An, Bình, Cường**.
- Các thao tác chèn/xóa/tìm kiếm đều là **$O(\log n)$** .
- Dùng tốt khi cần:
 - dữ liệu **không trùng lặp**,
 - **có thứ tự**,

- hỗ trợ truy vấn theo khoảng (*subset*, *tailSet*, *headSet*).

Việc sử dụng cây Đỏ-Đen trong TreeMap / TreeSet mang lại:

- **Hiệu năng ổn định**, không bị suy giảm như BST thường.
- **Thứ tự dữ liệu được duy trì liên tục**, phục vụ:
 - Hệ thống quản lý danh mục,
 - Cấu trúc index tạm thời,
 - Bộ nhớ đệm có thứ tự,
 - Các thuật toán xử lý dữ liệu theo khoảng.

→ Điều này cho thấy cây đỏ-đen là một trong những cấu trúc dữ liệu cốt lõi của Java Collections, được sử dụng rộng rãi trong mọi ứng dụng từ nhỏ đến lớn.

1.3. Ngôn ngữ khác

Một số thư viện của các ngôn ngữ khác cũng dùng cây đỏ-đen hoặc biến thể:

- **Python:** Thư viện bổ sung `sortedcontainers`, `rbtree`
- **Go:** Gói `container/rbtree` (không chính thức)
- **Rust:** Các crate như `rbtree`, hoặc dùng trong một số mô-đun của hệ thống runtime

👉 Những ngôn ngữ này cần một cấu trúc **ordered dictionary / ordered set** với hiệu năng ổn định, nên Red-Black Tree là lựa chọn quen thuộc.

2. Hệ Điều Hành Và Hệ Thống Nhân Kernel

Red-Black Tree được dùng trong nhiều thành phần của **Linux Kernel**, ví dụ:

- Quản lý bộ nhớ ảo (Virtual Memory Area – VMA)
- Lập lịch I/O
- Theo dõi timer (Linux timer wheel)
- Quản lý các cấu trúc vùng nhớ của tiến trình

👉 Ưu điểm: Tìm kiếm/ghi/xoá vùng nhớ nhanh, ổn định, chịu tải lớn.

3. Cơ Sở Dữ Liệu Và Hệ Thống Lưu Trữ

Một số hệ quản trị cơ sở dữ liệu sử dụng Red-Black Tree để quản lý:

- Chỉ mục đơn giản ở tầng bộ nhớ (in-memory index)
- Cấu trúc metadata
- Tập dữ liệu nhỏ hoặc tạm thời cần tìm kiếm nhanh

👉 Dù B-Tree/B+Tree phổ biến hơn trên đĩa, Red-Black Tree lại hiệu quả trong các chỉ mục **hoạt động trong RAM**.

4. Mạng Máy Tính Và Bảng Định Tuyến

Trong các thiết bị mạng, bảng định tuyến (routing table) thường có số lượng lớn mục cần tra cứu nhanh.

- Một số bộ định tuyến (routers) dùng Red-Black Tree để lưu bảng tiền tố (prefix table)
- Hỗ trợ lookup địa chỉ với tốc độ ổn định
- Thời gian cập nhật/tra cứu tránh rơi vào trường hợp xấu

5. Trình Biên Dịch Và Phân Tích Mã Nguồn

Red-Black Tree được dùng để quản lý:

- Bảng ký hiệu (symbol table)
- Tập biến, hàm và phạm vi (scope management)
- Các cấu trúc cần sắp xếp động theo khoá

Các trình biên dịch như GCC, LLVM có những module sử dụng RB-Tree hoặc biến thể của nó.

6. Trình Duyệt Web Và Công Cụ Render

Các trình duyệt như Chrome/Chromium dùng Red-Black Tree trong:

- Quản lý DOM Tree mapping
- Cache stylesheet
- Quản lý các đối tượng layout cần sắp xếp theo khoá

Các thao tác cập nhật, xoá và chèn liên tục khiến RB-Tree là lựa chọn hợp lý nhất.

VI. KẾT LUẬN

Qua quá trình nghiên cứu và phân tích, có thể thấy rằng **cây Đỏ-Đen (Red-Black Tree)** là một trong những cấu trúc dữ liệu cân bằng động quan trọng và được ứng dụng rộng rãi trong khoa học máy tính cũng như trong các hệ thống phần mềm hiện đại. Nhờ duy trì được chiều cao của cây luôn nhỏ hơn hoặc bằng $2 \times \log_2(n+1)$, các thao tác tìm kiếm, chèn và xoá đều đạt độ phức tạp thời gian $O(\log n)$ ngay cả trong trường hợp xấu nhất. Điều này đảm bảo hiệu năng ổn định và đáng tin cậy — yếu tố đặc biệt quan trọng đối với các hệ thống lớn, nơi dữ liệu thay đổi liên tục và yêu cầu tốc độ truy xuất nhanh.

Một ưu điểm nổi bật của Red-Black Tree so với các cấu trúc cân bằng khác như AVL Tree là số lượng phép quay (rotation) trong quá trình cân bằng lại ít hơn, nhờ cơ chế cân bằng “vừa đủ”. Đây là lý do khiến Red-Black Tree thường được ưu tiên trong các thư viện chuẩn, các cơ sở dữ liệu và nhiều hệ thống lớn. Bên cạnh đó, cấu trúc và quy tắc vận hành của cây đỏ-đen tương đối đơn giản khi triển khai trong thực tế, đồng thời hoạt động tốt ngay cả với dữ liệu ngẫu nhiên hoặc tăng dần — những trường hợp mà một số cấu trúc khác có thể gặp vấn đề.

Tuy vậy, Red-Black Tree vẫn tồn tại một số hạn chế nhất định. Việc cài đặt phức tạp hơn đáng kể so với BST thông thường do phải xử lý nhiều điều kiện xoay cây và thay đổi màu. Điều này khiến quá trình trực quan hóa, kiểm thử và gỡ lỗi trở nên khó khăn hơn. Hơn nữa, Red-Black Tree không đạt mức cân bằng hoàn hảo như AVL Tree, do đó một số thao tác trong những tình huống đặc biệt có thể chậm hơn. Ngoài ra, nó không phải lựa chọn tối ưu cho truy cập tuần tự vì mỗi node phải lưu thêm thông tin màu, và cấu trúc dạng cây luôn chậm hơn các cấu trúc tuyến tính nếu truy xuất theo thứ tự liên tục.

Mặc dù có những nhược điểm như vậy, **Red-Black Tree vẫn được đánh giá là một trong những cấu trúc dữ liệu phổ biến, ổn định và đáng tin cậy nhất**. Khả năng duy trì hiệu năng ở quy mô lớn và sự đơn giản tương đối trong triển khai thực tiễn khiến nó trở thành lựa chọn mặc định trong nhiều thư viện chuẩn, chẳng hạn như STL của C++, Java Collections Framework, và môi trường hệ thống của nhiều cơ sở dữ liệu và trình biên dịch.

Có thể khẳng định rằng: **“Điểm yếu lớn nhất của Red-Black Tree là độ phức tạp trong cài đặt và bảo trì; nhưng đổi lại, ta có được một cấu trúc cực kỳ ổn định, hiệu quả và phù hợp cho các hệ thống quy mô lớn.”**

Do đó, Red-Black Tree đã, đang và sẽ tiếp tục giữ vai trò quan trọng trong việc xây dựng các cấu trúc dữ liệu cốt lõi, đặc biệt trong bối cảnh lượng dữ liệu ngày càng tăng và yêu cầu xử lý thời gian thực ngày càng cao.

VII. KẾ HOẠCH GIAI ĐOẠN 2

Sau khi hoàn thành việc nghiên cứu và trình bày chi tiết về **Red-Black Tree** trong giai đoạn 1, kết quả cho thấy đây là một cấu trúc dữ liệu cân bằng hiệu quả khi làm việc trong bộ nhớ RAM và được ứng dụng rộng rãi trong các thư viện lập trình (như `std::map`, `TreeMap`,...). Tuy nhiên, Red-Black Tree chưa phải là lựa chọn tối ưu khi lượng dữ liệu rất lớn và việc truy cập phụ thuộc vào **bộ nhớ ngoài (disk/SSD)** – vốn là đặc trưng quan trọng của các hệ quản trị cơ sở dữ liệu và hệ thống lưu trữ ngày nay.

Vì vậy, trong **giai đoạn 2**, em lựa chọn mở rộng nghiên cứu sang **B-Tree** – một cấu trúc dữ liệu tổng quát hơn, có khả năng xử lý dữ liệu lớn trên bộ nhớ thứ cấp và được xem là nền tảng của nhiều hệ thống hiện đại như **database, filesystem, indexing structure**,...

Mục tiêu giai đoạn 2:

1. Tìm hiểu lý thuyết B-Tree

- Khái niệm và bài toán đặt ra khi dữ liệu được lưu trên disk.

- Cách tổ chức node với nhiều khóa và nhiều con.
- Điều kiện của một cây B-Tree trong mô hình chuẩn (bậc t).
- 2. **Phân tích sự khác biệt giữa Red-Black Tree và B-Tree**
 - Lý do tại sao B-Tree được dùng cho cơ sở dữ liệu.
 - Vì sao độ cao của B-Tree thấp hơn đáng kể.
 - Tối ưu số lần truy cập đĩa (disk I/O).
- 3. **Xây dựng và mô phỏng các thao tác chính trên B-Tree**
 - Tìm kiếm (Search)
 - Chèn (Insertion) kèm theo cơ chế tách node (split)
 - Xóa (Deletion) kèm theo cơ chế gộp hoặc vay khóa (merge/borrow)
- 4. **Cài đặt thử nghiệm**
 - Viết code minh họa thao tác Insert/Search/Delete.
 - Xây dựng ví dụ trực quan và so sánh số phép đọc/ghi đĩa ước lượng.
- 5. **Phân tích ứng dụng thực tế**
 - Hệ quản trị cơ sở dữ liệu (MySQL, PostgreSQL).
 - Chỉ mục (index) trong database.
 - File system (NTFS, HFS+, EXT4).
 - Các hệ thống lưu trữ quy mô lớn.

Kết quả mong đợi:

- Hiểu rõ cách B-Tree khắc phục hạn chế của Red-Black Tree trong môi trường dữ liệu lớn.
- Nắm bắt nguyên lý tổ chức dữ liệu tối ưu trên bộ nhớ thứ cấp.
- Có khả năng giải thích tại sao gần như mọi hệ thống database sử dụng B-Tree hoặc biến thể của B-Tree.
- Hoàn thành bài trình bày cuối kỳ để tất cả mọi người đều hiểu rõ B-Tree

VIII. TÀI LIỆU THAM KHẢO

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, *Giới thiệu về Giải thuật (Introduction to Algorithms)*, 3rd Edition, The MIT Press, 2009.
2. Tài liệu bài giảng môn *Cấu trúc dữ liệu & Giải thuật*, Đại học Bách Khoa Hà Nội.
3. Website tham khảo:
 - Visualgo – [Red-Black Tree Animation](#)
 - Programiz – [Red-Black Tree](#)
 - Wikipedia – [Red-Black Tree](#)